

OptaDOS: User Guide

Version 1.3.388

Andrew J. Morris, Rebecca J. Nicholls, Chris J. Pickard, Jonathan R. Yates

Department of Metallurgy and Materials
University of Birmingham
Elms Road
Edgbaston
Birmingham
UK

Department of Materials
University of Oxford
Parks Road
Oxford
UK

and

Department of Materials Science and Metallurgy
University of Cambridge
27 Charles Babbage Rd
Cambridge
UK

March 26, 2026

For the Photoemission Module:

Felix Mildner, Victor Chang Lee, Bruno Camino, Nicholas M. Harrison

Department of Materials
Imperial College London
Royal School of Mines
South Kensington Campus
London SW7 2AZ
UK

and

Department of Chemistry
Imperial College London
South Kensington Campus
London SW7 2AZ
UK

University of Cambridge, University of Oxford and University College London

© 2013-2026 A. J. Morris, R. J. Nicholls, C. J. Pickard and J. R. Yates.

First published 2010
This edition 2026

10 9 8 7 6 5 4 3 2 1

Part of this work published in:

“OptaDOS: A Tool for Obtaining Density of States, Core-loss and Optical Spectra from Electronic Structure Codes”, Andrew J. Morris, Rebecca J. Nicholls, Chris J. Pickard and Jonathan R. Yates, *Comp. Phys. Comm.* **185**, 5, 1477 (2014)

Further information about OPTADOS may be obtained from:
www.optados.org

Nota bene: OPTADOS, optados or OptaDOS, but never OPTADOS.

Font: Computer Modern, 11pt
Typeset by the authors using L^AT_EX

Contents

1	Introduction	9
1.1	Background	9
1.2	Features	9
2	Theoretical Background	11
2.1	Integrals over the Brillouin Zone	11
2.2	Brillouin Zone Sampling Schemes	11
2.3	Density of Electronic States	13
2.4	Dielectric response	13
2.5	Photoemission model	16
3	Getting Started	25
3.1	Installation	25
3.2	Usage	26
4	Structure of the Program	27
4.1	Structure of the Photoemission module	29
5	Parameters	33
5.1	seedname.odi File	33
5.2	General Parameters	34
5.3	DOS Parameters	37
5.4	JDOS Parameters	38
5.5	PDOS Parameters	39
5.6	Projected Dispersion Parameters	39

5.7	Optics Parameters	40
5.8	Core-level Parameters	41
5.9	Photoemission Parameters	42
6	CASTEP Examples	49
6.1	Density of States	49
6.2	Projected Density of States	53
6.3	Projected Bandstructure	55
6.4	JDOS	56
6.5	OPTICS	56
6.6	CORE	58
6.7	CORE with Mizoguchi Chemical Shift	61
6.8	Photoemission	63
7	ONETEP Examples	67
7.1	(L)PDOS	67
7.2	EELS/ELNES	68
8	Frequently Asked Questions	69
8.1	OPTADOS crashes complaining that it can't read the seed.bands or seed.cst_ome file.	69
8.2	The examples do not work and I'm using CASTEP version 6.0	69
8.3	I'm getting odd results for my PDOS calculation when using CASTEP version 6.0	69
8.4	OPTADOS is reporting the wrong number of kpoints	70
8.5	I'd like OPTADOS to do X as well	70
8.6	I'd like to help, what can I do?	70
8.7	I think I've found a bug: what should I do?	70
A	Interface with other codes	71
A.1	.bands file	71
A.2	.ome_bin file	73
A.3	.pdos_bin file	74
A.4	.elnes_bin file	76

A.5	.dome_bin file	77
A.6	.fem_bin file	78
A.7	.tmprob_bin file	79
A.8	.gkgrid_bin file	80

We would like to thank Phil Hasnip and Keith Refson for very helpful discussions and Gareth Griffiths and Kane O'Donnell for extensive beta testing. AJM, RJN and CJP acknowledge support from the EPSRC. Additionally AJM acknowledges the Winton programme for the physics of sustainability and RJN the ERC (Grant ERC-2009-StG-240500). JRY acknowledges support from the Royal Society through a University Research fellowship.

AJM, RJN, CJP and JRY
September 2013

Contributors

- Matthew L. Evans, University of Cambridge: projected bandstructures and bug fixes.
- Angela F. Harper, University of Cambridge: Mizoguchi chemical shift for coreloss calculations.

Chapter 1

Introduction

1.1 Background

OPTADOS is a code for calculating optical, core-level excitation spectra along with full, partial and joint electronic density of states (DOS). The code was developed by merging the `LinDOS` code of Andrew Morris and Chris Pickard at University College London with the optical properties code of Rebecca Nicholls and Jonathan Yates at Oxford University. OPTADOS is written in Fortran 95 and may be run in parallel using `MPI`. At present OPTADOS interfaces with `CASTEP` and `ONETEP` output files, although it is extendible to perform calculations on any set of band eigenvalues and their derivatives generated by any electronic structure code.

The code is freely available through the GPL licence with the request that the following citation (quoted in full) is required in any publication resulting from the use of OPTADOS.

*Andrew J. Morris, R. J. Nicholls, C. J. Pickard and Jonathan R. Yates, OPTADOS: A Tool for Obtaining Density of States, Core-level and Optical Spectra from Electronic Structure Codes, Comp. Phys. Comm. **185**, 5, 1477, (2014)*

Further information and examples can be found at

www.optados.org

1.2 Features

OPTADOS generates optical, core-level excitation spectra along with full, partial and joint electronic DOS. The DOS, PDOS and JDOS take advantage of the linear and adaptive broadening schemes which are more accurate than standard Gaussian broadening since they exploit knowledge of the gradients of the bands at each k -point in the Brillouin zone. These DOS are the basis of the more advanced functionality of OPTADOS, the core and optical spectra.

Along with data text files OPTADOS also generates `.agr` files of results to be read by `grace`.

Chapter 2

Theoretical Background

2.1 Integrals over the Brillouin Zone

Many energy-resolved spectral properties of a material take the form of an integration of some function, $F_{nm}(\mathbf{k})$ in reciprocal space over the BZ; where $F_{nm}(\mathbf{k})$ are the elements of a periodic operator which commutes with the crystal translational symmetry. Perhaps the two simplest examples of integrals, hereinafter referred to as type (a) and type (b), take the form at $T = 0$ K of,

$$I^{(a)}(E) = \sum_n \int_{\text{BZ}} \frac{d\mathbf{k}}{(2\pi)^3} F_{nn}(\mathbf{k}) \delta(E_{n\mathbf{k}} - E), \quad (2.1)$$

and,

$$I^{(b)}(E) = \sum_n^{\text{occ}} \sum_m^{\text{unocc}} \int_{\text{BZ}} \frac{d\mathbf{k}}{(2\pi)^3} F_{nm}(\mathbf{k}) \delta(E - (E_{m\mathbf{k}} - E_{n\mathbf{k}})), \quad (2.2)$$

where $E_{n\mathbf{k}}$ are the energy eigenvalues of the system. They are also extendible to integration over spin-polarised channels in a straightforward way.

2.2 Brillouin Zone Sampling Schemes

Since the crystal momentum, $\mathbf{k}$, is a continuous quantum number the band structure in an *ab initio* calculation is approximated using specific k -points within the first Brillouin zone. These k -points may be chosen through symmetry perhaps using a Monkhorst-Pack grid [11] or the multi- k -point generalisation to the Baldereschi scheme [18, 12].

Obtaining the energy bands from a set of eigenenergies at discrete k -points is non-trivial. This simplest way of assigning bands, by counting eigenvalues at k -points, implicitly assumes that there are no band crossings in the BZ. This is further complicated by “band kissing” where bands approach, but due to small interactions between the bands breaking the degeneracy they remain continuous [2, 16]. Considering band coefficients and the overlap matrix it has been shown how to assign bands to individual eigenenergies [25, 19], however these still require a broadening scheme to obtain a DOS. Below we detail some approaches to obtaining a DOS from a set of eigenenergies.

2.2.1 Gaussian broadening

The reciprocal space is discretised into *sub-cells* each containing a k -point which contributes to the integral corresponding to the energy of the k -point at their centre. Band dispersion may be crudely approximated by smearing each sub-cell's contribution by a fixed-width Gaussian function, of some width, ω ,

$$\delta(E_{n\mathbf{k}} - E) \rightarrow \frac{1}{\omega\sqrt{2\pi}} \exp\left(-\frac{1}{2}((E_{n\mathbf{k}} - E)/\omega)^2\right). \quad (2.3)$$

This approach requires a large number of k -points to converge the DOS since when choosing ω there is a trade-off between representing the sharp features, such as van Hove singularities[23] and not introducing spurious oscillations due to the limited k -point sampling. The band crossing problem is not present since the bands are modelled as dispersionless.

Fixed-width Gaussian broadening may be chosen for OPTADOS calculations with, **broadening : fixed.**

2.2.2 Interpolative schemes

In the linear tetrahedron interpolative scheme the BZ is divided into roughly equal size tetrahedra with the band energy calculated at each vertex [7]. The DOS contribution of each tetrahedron is then calculated analytically by linear interpolation.

Linear interpolation suffers from the band crossing problem, since the interpolation always joins the lowest energy bands together, which has the effect of avoiding all band crossing and kissing. The method also does not describe van Hove singularities well since the interpolation is linear.

Higher-order interpolation schemes, such as the quadratic tetrahedron scheme, have been proposed which converge the singularities more rapidly than the linear scheme [3, 8, 9], but these still require a high-density k -point grid to mitigate the band-crossing problem.

2.2.3 Extrapolative schemes

Extrapolating from each k -point rather than interpolating between adjacent points eliminates the band-crossing problem. Müller and Wilkins show that for free electron bands the DOS is a smooth parabola when using the extrapolative method and a rough unconverged spectrum with the interpolative approach when using the same k -point mesh [13].

The linear extrapolative scheme still does not represent van Hove singularities as well as a second-order approach. Band curvatures may be combined with Löwdin perturbation theory to extrapolate to higher order [16, 17].

Linear extrapolative broadening may be chosen for OPTADOS calculations with, **broadening : linear.**

2.2.4 Adaptive broadening schemes

An approximation to linear extrapolation method was proposed by Yates *et al.* [24]. Each sub-cell's contribution to the DOS is broadened by a Gaussian function whose width, ω , is proportional to the energy band gradient as its k -point. This simple approach removes the spurious oscillations of the fixed broadening whilst maintaining the sharper features. The adaptive broadening scheme was introduced in Ref [24] in the context of Wannier interpolation of BZ quantities. However, the scheme is more general than this, and with OPTADOS we apply adaptive broadening directly to the quantities obtained from the electronic structure calculation.

Adaptive broadening may be chosen for OPTADOS calculations with, `broadening : adaptive`.

2.3 Density of Electronic States

When $F_{nm}(\mathbf{k}) = 1$ the type (a) integral represents the electronic density of states (DOS) at E and the total ground-state energy, E_{gs} is obtained by integrating over the occupied energies,

$$E_{gs} = \int_0^{E_F} I^{(a)}(E; F_{nm}(\mathbf{k}) = 1) dE, \quad (2.4)$$

where E_F is the Fermi energy.

The DOS is plotted with OPTADOS using the `task : dos` command.

The partial, or projected DOS (PDOS), I_μ is obtained by projecting the DOS onto local orbitals, μ , at each atomic site[21]. In this case,

$$F_{\mu n}(\mathbf{k}) = \Re \left\{ T_{\mu n}^\dagger(\mathbf{k}) T_{\nu n}(\mathbf{k}) S_{\nu \mu}(\mathbf{k})^{-1} \right\}, \quad (2.5)$$

where $T_{\mu n}(\mathbf{k}) = \langle \Psi_n(\mathbf{k}) | \phi_\mu(\mathbf{k}) \rangle$ are the overlap matrices between LCAO (linear combination of atomic orbitals) basis, ϕ_μ and plane-wave states, Ψ_n , and $S_{\mu n}(\mathbf{k}) = \langle \phi_n(\mathbf{k}) | \phi_\mu(\mathbf{k}) \rangle$, the overlap matrices of the LCAO basis.

The PDOS is plotted with OPTADOS using the `task : pdos` command. The projection is defined using the `pdos` keyword as described in the user manual.

In the case of the type (b) integral when $F_{nm}(\mathbf{k}) = 1$, we obtain the joint-DOS, (JDOS), for frequency, ω . The JDOS is plotted with OPTADOS using the `task : jdoss` command.

2.4 Dielectric response

Core-level spectra and optical properties are related to the complex dielectric function $\varepsilon_1 + i\varepsilon_2$. Within the random phase approximation, the imaginary part of the dielectric function is

$$\varepsilon_2(\omega) = \frac{\pi e^2}{\varepsilon_0 \Omega} \frac{1}{q^2} \left| \langle \psi_{\mathbf{k}}^c | e^{-i\mathbf{q} \cdot \mathbf{r}} | \psi_{\mathbf{k}+\mathbf{q}}^v \rangle \right|^2, \quad (2.6)$$

where e is the charge on an electron, ϵ_0 is the permittivity of free space, Ω is the volume of the unit cell and $\mathbf{q}$ is the momentum transfer [4]. Using the dipole approximation, expressing $\mathbf{q}$ as $q \hat{\mathbf{u}}$ and taking the limit that $q \rightarrow 0$, ϵ_2 can be written as an integral of type (b),

$$\epsilon_2(\omega) = \frac{\pi e^2}{\epsilon_0 \Omega} I^{(b)}(\hbar\omega), \quad (2.7)$$

where,

$$F_{nm}(\mathbf{k}) = |\langle \psi_{\mathbf{k}}^c | \hat{\mathbf{u}} \cdot \mathbf{r} | \psi_{\mathbf{k}}^v \rangle|^2. \quad (2.8)$$

Most electronic structure codes do not calculate $\langle \psi_{\mathbf{k}}^c | \mathbf{r} | \psi_{\mathbf{k}}^v \rangle$ directly due to the ambiguity of a position operator in periodic cells. Rather they calculate $\langle \psi_{\mathbf{k}}^c | \mathbf{p} | \psi_{\mathbf{k}}^v \rangle$, where $\mathbf{p}$ is the momentum operator, which is trivial in reciprocal space.

Since,

$$\mathbf{p} = m\dot{\mathbf{r}} = \frac{im}{\hbar} [\hat{H}, \mathbf{r}] \quad (2.9)$$

where $\hat{H}$ is the Hamiltonian operator and m the electron mass, it follows that:

$$\langle \psi_{\mathbf{k}}^c | \mathbf{r} | \psi_{\mathbf{k}}^v \rangle = \frac{\hbar}{im(e^c - e^v)} \langle \psi_{\mathbf{k}}^c | \mathbf{p} | \psi_{\mathbf{k}}^v \rangle \quad (2.10)$$

where e^c and e^v are the energy eigenvalues of the states $|\psi_{\mathbf{k}}^c\rangle$ and $|\psi_{\mathbf{k}}^v\rangle$. Note that had we added scissor operator term to our Hamiltonian $\hat{H} = \hat{H}_{KS} + \Delta_{\text{scissor}}$ then this also appears in the conversion from momentum matrix elements to optical matrix elements.

Once ϵ_2 is known, it is possible to use the Kramers-Kronig relations to obtain ϵ_1 and hence obtain an expression for the full dielectric function. The dielectric function is a tensor and in OPTADOS it is either possible to output the full dielectric tensor or calculate dielectric function for the isotropic average of $\mathbf{q}$, a particular direction of $\mathbf{q}$ or the average of $\mathbf{q}$ over a plane perpendicular to a given direction. Once the dielectric function has been calculated, core-level spectra and optical properties can be calculated.

Core-level and low-loss EELS spectra are all related to the loss function, which is defined as [5]:

$$\Lambda = \Im \left(\frac{-1}{\epsilon_1 + i\epsilon_2} \right). \quad (2.11)$$

2.4.1 Core-level spectra

For core-level spectroscopy (core-loss EELS and x-ray absorption near edge spectroscopy (XANES)) the incident perturbation is far from resonance and since $\epsilon_1 \approx 1$ and $\epsilon_2 \gg 1$ equation 2.11 reduces to $\Lambda = \epsilon_2$ [5]. The type (b) integral in equation 2.7 becomes a type (a) integral as the energy of the core state is fixed. Emission spectra (transitions from valence to core XES) can also be calculated by considering a sum over the occupied rather than unoccupied states. Core-level spectra are calculated in OPTADOS using the `task : core` command.

Several different experimental geometries are possible for core-level experiments and OPTADOS allows the calculation of spectra with $\mathbf{q}$ in a particular direction or an isotropic average.

In core-level absorption spectra there are several sources of broadening coming from the experimental set up and lifetime effects. These can be included in OPTADOS by broadening the theoretical spectrum using a combination of Gaussian and Lorentzian functions. To include instrumentation and lifetime effects, `core_LAI_broadening` should be set to true.

2.4.2 Optical properties

In the low-loss EELS regime, the approximations used for core-level spectroscopy do not hold and the full form of the loss function needs to be calculated. This is done by calculating ε_2 using equations 2.7 and 2.8 and then using the Kramers-Kronig relations to find ε_1 . Once the dielectric function has been calculated, the loss-function is simulated (without local field effects) using equation 2.11. As the dielectric function has been calculated, several other optical properties (which are listed below) can also be computed [4].

Conductivity, in units of Sm^{-1} , is computed using:

$$\sigma_1 = \varepsilon_0 \varepsilon_2 \omega, \quad (2.12)$$

$$\sigma_2 = \varepsilon_0 (1 - \varepsilon_1) \omega. \quad (2.13)$$

The refractive index is calculated using:

$$N = n + ik \quad (2.14)$$

where n and k are obtained from,

$$n^2 = \frac{1}{2}([\varepsilon_1^2 + \varepsilon_2^2]^{\frac{1}{2}} + \varepsilon_1) \quad \text{and} \quad k^2 = \frac{1}{2}([\varepsilon_1^2 + \varepsilon_2^2]^{\frac{1}{2}} - \varepsilon_1). \quad (2.15)$$

The absorption coefficient, (in units of m^{-1}), is computed using

$$\eta = \frac{2k\omega}{c}, \quad (2.16)$$

and reflection coefficient, R , from,

$$R = \frac{(n - 1)^2 + k^2}{(n + 1)^2 + k^2}. \quad (2.17)$$

All optical properties are calculated in OPTADOS using the `task : optics` command.

2.4.3 Intraband term

For metals it is necessary to consider an intraband contribution to the optical response, i.e. a term when $n = m$ in equation 2.2 [4, 1]. The contribution to ε can be written as a type (a) integral with $\hbar\omega$ set to E_F ,

$$\varepsilon_1^{\text{intra}}(\omega) = 1 - \frac{e^2}{\varepsilon_0 \Omega} \frac{1}{(\omega^2 + \Gamma^2)} I^{(a)}(E_F), \quad (2.18)$$

$$\varepsilon_2^{\text{intra}}(\omega) = \frac{e^2}{\varepsilon_0 \Omega} \frac{\Gamma}{\omega(\omega^2 + \Gamma^2)} I^{(a)}(E_F), \quad (2.19)$$

where Γ denotes the relaxation rate.

2.5 Photoemission model

The formalism adopted in the photoemission model adopted here requires that the excitation spectrum can be described within a quasi-particle model, with excitations represented as transitions between energy levels in a band structure of single particle states. The one-particle transition probability for the absorption of a photon of frequency ω is defined by Fermi's golden rule,

$$\Gamma_{i \rightarrow j} = \frac{2\pi}{\hbar} M(i, j, \mathbf{k}, s) \delta(E(j, \mathbf{k}, s) - E(i, \mathbf{k}, s) - \hbar\omega) \quad (2.20)$$

where E is the electronic state eigenvalue, i and j are the indices for the initial and final state respectively, s is the spin index, $\mathbf{k}$ is the wave vector of the electronic wave function, $M(i, j, \mathbf{k}, s)$ is the photoemission matrix element and the Dirac delta function $\delta(E(j, \mathbf{k}, s) - E(i, \mathbf{k}, s) - \hbar\omega)$ ensure the energy conservation in the photoemission process. The Dirac delta function can be evaluated using the fixed-width Gaussian broadening, adaptive Gaussian broadening or linear extrapolative scheme implemented in the OptaDOS code.

The photoemission matrix element $M(i, j, \mathbf{k}, s)$ in Eq. 2.20 is

$$M(i, j, \mathbf{k}, s) = |\langle \psi_{\mathbf{k}, j, s} | H' | \psi_{\mathbf{k}, i, s} \rangle|^2 \quad (2.21)$$

The Fermi's golden rule describes the transition probability from an initial eigenstate ψ_i to a final state ψ_j as a result of a weak perturbation H' cause by the light. The perturbation due to the light H' can be evaluated by calculating the effect of the electromagnetic field on the electron (Eq. 2.22).

$$H' = \frac{e}{2m_0} (p \cdot A + A \cdot p) + \frac{e^2 A^2}{2m_0} \quad (2.22)$$

where $p = -i\hbar\nabla$ is the momentum operator and here A is the polarization vector of the incoming electromagnetic wave. The A^2 term can be neglected under the assumption of a weak field. The Hamiltonian can be simplified due to the Coulomb gauge, $\nabla \cdot A = 0$. The perturbation due to the light field is rewritten as

$$H' = \frac{e}{2m_0} p \cdot A \quad (2.23)$$

The optical matrix from which the optical properties of a solid can be obtained.

2.5.1 Photoemission final state

Within the one-electron photoemission approach, two distinct photoemission models have been developed and widely used in previous work in order to describe the photoemission process. The current implementation generates two distinct optical matrices (Eq. 2.21) where the final state ψ_j changes according to the photoemission model.

- **Three step photoemission model** : The three step model divides the photoemission process into three steps. In the first step, the ground state electrons are excited into a final state in the crystal or Bloch state

$$\psi_j(\mathbf{r}) = \sum C_{j,\mathbf{G}} e^{i(\mathbf{k}+\mathbf{G})\mathbf{r}} \quad (2.24)$$

Where $C_{j,\mathbf{G}}$ are the plane wave coefficients and $\mathbf{G}$ are the reciprocal lattice vectors. In the second step, the excited Bloch electrons will have a probability to travel to the surface of the material. Once the electrons reach the surface of the material, in the third step, the electrons will escape into the vacuum.

- **One step photoemission model** : The one step model final state in this work is described as

$$\psi_j(\mathbf{r}) = e^{i\mathbf{k}\mathbf{r}} e^{-\frac{z \cdot \sin(1-\theta(i,j,\mathbf{k}))}{\lambda(\omega)}} \quad (2.25)$$

where $\lambda(\omega)$ is the electrons scattering length and $\sin(1 - \theta(i, j, \mathbf{k}))$ considers the scattering length increase with emission angles away from the surface normal.

2.5.2 Atomic projection

It is possible to project the energy bands of a periodic solid onto localised atom centred orbitals, and therefore to define atomic contributions to the excitation. This is particularly convenient as it allows to explicitly calculate the contributions from excitation in each sub-surface layer. The contribution of each atomic layer that composed the slab can be performed by projecting the delocalised Bloch functions onto localised representations such as atomic orbitals (AO) or Wannier functions. CASTEP calculates this projection $W(i, j, \mathbf{k}, \mu, \mathbf{s})$ as

$$W(i, j, \mathbf{k}, \mu, \mathbf{s}) = \sum_{\nu} \mathbf{T}_{\mu ij, \mathbf{s}}^*(\mathbf{k}) \mathbf{T}_{\mu ij, \mathbf{s}}(\mathbf{k}) \mathbf{S}_{\nu \mu}(\mathbf{k})^{-1} \quad (2.26)$$

where $T_{\mu ij, \mathbf{s}}(\mathbf{k}) = \langle \Psi_{ij, \mathbf{s}}(\mathbf{k}) | \phi_{\mu}(\mathbf{k}) \rangle$ are the overlap matrices between linear combination of atomic orbitals (LCAO) basis $\phi_{\mu}(\mathbf{k})$ and plane wave state $\Psi_{ij, \mathbf{s}}(\mathbf{k})$, and $S_{\nu \mu}(\mathbf{k}) = \langle \phi_{\nu}(\mathbf{k}) | \phi_{\mu}(\mathbf{k}) \rangle$ are the overlap matrices of the LCAO basis.

2.5.3 Temperature

The original formulation of the Kohn-Sham DFT describes a system of interacting particles in an external potential at zero temperature. Later, Mermin extended this formalism for a finite temperature $T > 0$. In the current implementation, the finite temperature formalism of Mermin is adopted by introducing a Fermi-Dirac distribution in which the state occupancy is

$$n(i, \mathbf{k}, T, s) = \frac{1}{e^{(E_F - E_i(\mathbf{k}, s))/k_B T} + 1} \quad (2.27)$$

where E_F is the Fermi energy and k_B is the Boltzmann constant.

2.5.4 Light decay

The intensity of the light at each atom μ of the surface, expressed as

$$I(\omega, \mu) = I(\omega, \mu - 1)e^{-\alpha(\omega, \mu)t(\mu)} - R(\omega, \mu) \quad (2.28)$$

where $\alpha(\omega, \mu)$ is the absorption coefficient within each layer, $t(\mu)$ is the length of the light path and $R(\omega, \mu)$ is the reflective coefficient of the surface. The calculation of the optical characteristics is analogous to the equations in Section 2.4.2, but the contributions from each layer is calculated by weighting the electronic structure by the weights W from Section 2.4.2.

2.5.5 Electron transport

Determining absorption in each sub-surface layer requires the use of an explicit electron transport model for each of the layers. In the current implementation, a simple model of the inelastic scattering rate is used for the electron transport in the three step model and the escape probability of a free electron as it travels through a material in the one step model. The inelastic mean free path theory explains that electrons can only travel a certain average distance without suffering an inelastic scattering event. The assumption that once an electron suffers an inelastic scattering event the electron will be reabsorbed is adopted in this work, resulting in an escape function for the $i \rightarrow j$ excitation as a function of the escape depth;

$$esc(\omega, i, j, \mathbf{k}, \mu, s) = e^{-\frac{d(\mu)\sin(1-\theta(i, j, \mathbf{k}, s))}{\lambda(\omega)}} \quad (2.29)$$

The angle $\theta(i, j, \mathbf{k})$ indicates that electrons that travels with a higher angle with respect to the surface normal travels a longer distance inside the material. When passing the potential "step" at the surface it is assumed that the electron's transverse momentum is conserved, while the longitudinal momentum is reduced by the work function barrier. Therefore two sets of angles must be calculated for within the material (θ_{int}) and after passing the surface (θ_{ext}). These angles are calculated as

$$\begin{aligned} \theta_{int}(i, j, \mathbf{k}, s) &= \arccos \frac{E_{\parallel}(i, \mathbf{k}, s)}{E_K(j, \mathbf{k}, s) + W} \\ \theta_{ext}(i, j, \mathbf{k}, s) &= \arccos \frac{E_{\parallel}(i, \mathbf{k}, s)}{E_K(j, \mathbf{k}, s)} \end{aligned} \quad (2.30)$$

where $E_{\parallel}(i, \mathbf{k})$ is the transverse energy, $E_K(j, \mathbf{k})$ is the final state kinetic energy and W is the work function. The angle θ_{ext} is used to determine if emission is possible under the chosen conditions, while θ_{int} is used to model the electron's propagation distance.

2.5.6 Transverse momentum conservation

The momentum of photons can be considered as negligible compared to the momentum of the surface electrons and therefore, the electron momentum is conserved upon excitation. In

addition, since electrons can only travel very short distances inside a material, photoemission is considered as a surface phenomenon. At the surface of a crystal, the periodic transverse symmetry perpendicular to the surface is broken and so the pseudo momentum perpendicular to the surface is not a conserved quantity. The parallel momentum conservation is described by

$$\Theta'(\mathbf{k}_\perp) \begin{cases} 1, & \mathbf{k}_\perp(i, j, s) \geq 0 \\ \delta((E_v - E(j, \mathbf{k}), s), & \mathbf{k}_\perp(i, j, s) < 0 \end{cases} \quad (2.31)$$

where E_v is the vacuum level and

$$k_\perp(i, j, \mathbf{k}, s) = \frac{2m}{\hbar^2} (E(j, \mathbf{k}, s) - E_v) - \mathbf{k}_\parallel(i, \mathbf{k}, s)^2 \quad (2.32)$$

Here $\mathbf{k}_\parallel(i, \mathbf{k}, s)$ is the electron parallel momentum inside the crystal.

Θ' is a modified version of the Heavyside step function Θ in which 0 is replaced by a Dirac delta function $\delta(E_v - E_f)$ evaluated with a Gaussian function is included to account for finite temperature broadening of electrons that do not strictly fulfil the parallel momentum conservation.

$$\delta(E_v - E(j, \mathbf{k}, s)) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{1}{2} \frac{E_v - E(j, \mathbf{k}, s)}{\sigma^2}\right) \quad (2.33)$$

where the width σ can be used to approach external parameters such as the electron temperature.

2.5.7 Emission from the surface

In the three step model the final step considers the transmission across the surface into vacuum. The approach in the OptaDOS photoemission module is based on S. Hüfner, in *Advanced Texts in Physics*, Springer Berlin, Heidelberg, 3 edn., 2003, ch. 6, pp. 349-357.

To approximate the probability of this occurring, one can represent the propagating electron using a set of plane waves. The final bands in the three step model are bloch bands, so they can be represented as

$$\psi_f(\mathbf{k}) = \sum_G u_f(\mathbf{k}, \mathbf{G}) e^{i(\mathbf{k}+\mathbf{G})\cdot\mathbf{r}} \quad (2.34)$$

where u_f are the plane wave coefficients, $\mathbf{G}$ are the reciprocal lattice vectors, $\mathbf{k}$ is the electron momentum, and $\mathbf{r}$ is the position vector. Each of these components is able to match onto an equal component on the other side of the surface. Since the the transverse momentum is conserved, all the components with the same $\mathbf{k}_\parallel + \mathbf{G}_\parallel$ must be treated together. Thus the total transmission probability can be approximated as

$$|T(E_f, \mathbf{k}_{||})|^2 = |t(E_f, \mathbf{k}_{||})|^2 \left| \sum_{(k+G)_{\perp} > 0} u_f(\mathbf{G}, \mathbf{k}) \right|^2 \quad (2.35)$$

where $|t(E_f, \mathbf{k}_{||})|^2$ represents the transmission probability for the electron in general and the sum $\sum_{(k+G)_{\perp} > 0}$ is the contribution from all the plane wave components propagating towards the surface. The factor $|t(E_f, \mathbf{k}_{||})|^2$ is already included in Equation (2.31), so only the second factor is explicitly calculated in CASTEP and supplied to OptaDOS in a `*.tmprob_bin` file. Thus we can define

$$T(E_f, \mathbf{k}_{||}) = \left| \sum_{(k+G)_{\perp} > 0} u_f(\mathbf{G}, \mathbf{k}) \right|^2 \quad (2.36)$$

2.5.8 Field emission

In applications such as the generation of free electrons for particle accelerators, an external electric field is commonly used to increase the photocathode's efficiency for a given light energy. The effect of this external electric field is included using an approximation based on the Fowler-Nordheim (FN) theory of field emission. The external electric field is assumed to be both uniform and to only modify the vacuum potential barrier, the electron distribution is in thermodynamic equilibrium and obeys Fermi-Dirac statistics and the emission surface is flat and planar with a constant uniform local work function. The barrier height is lowered due to the presence of the electric field. This lowering is calculated as

$$W_{eff} = W - \sqrt{\frac{e^3 F}{4\pi\epsilon_0}} \quad (2.37)$$

where W is the user supplied work function, F is the electric field strength in V/m and ϵ_0 is the vacuum permittivity. The vacuum potential in the FN theory in this current implementation is described using a Schottky-Nordheim barrier M defined as

$$M = \phi - eFz - \frac{e^2}{16\pi\epsilon_0 z} \quad (2.38)$$

where e is the electron charge, ϵ_0 is the vacuum permittivity, ϕ is the work function barrier, F is the external electric field and z is the distance from the surface. The current implementation follows the arguments put forward in an article by Forbes [6] to calculate the transmission probability across the modified barrier.

The transmission probability D is calculated as

$$D \approx \exp[-G] \quad (2.39)$$

which assumes that the so-called "JWKB exponent" G is sufficiently large that $\exp[-G] \ll 1$. G is generally defined as

$$G \equiv g_e \int_{z_1}^{z_2} M^{\frac{1}{2}} dz \quad \text{with} \quad g_e = \frac{2(2m_e)^{1/2}}{\hbar} \quad (2.40)$$

where the integral is over the range with $M > 0$, i.e. z_1 and z_2 , m_e is the electron mass and $\hbar$ is reduced Planck's constant.[6] The integral boundaries are calculated as turning points of M by using

$$z_{2,1} = \frac{\phi \pm \sqrt{\phi^2 - \frac{e^3 F}{4\pi\epsilon_0}}}{2eF} \quad (2.41)$$

where ϕ is the barrier height of the electron in J , e is the electron charge in C , F is the electric field in V/m , and ϵ_0 is the vacuum permittivity in . The numerical integration for each band's is performed using a recursive bisection method, where each segment's contribution is approximated using Simpson's rule.

2.5.9 Projected, bulk and total quantum efficiency and mean transverse energy

For the three step model the QE can be thus defined by the following. By considering Fermi's Golden rule (Eq. 2.20) for the excitation probability, the projection onto atomic orbitals (Eq. 2.26), temperature effects on the electronic population (Eq. 2.27), light decay (Eq. 2.28), electron scattering effects (Eq. 2.29), the momentum conservation rules (Eq. 2.31), and the probability for transmission across the surface (Eq. 2.36), the QE in this model is computed as

$$QE(\omega, \mathbf{k}, \mu, T, s) = \frac{1}{A} \sum_{i,j,s} \Gamma_{i \rightarrow j} W(i, j, \mathbf{k}, \mu, s) n(i, T, s) (1 - n(j, T, s)) I(\omega, \mu) \\ esc(\omega, f, \mathbf{k}, \mu, s) \Theta(\mathbf{k}_\perp) T(E_f, \mathbf{k}_\parallel) (1 + D(j, s)) \quad (2.42)$$

where A is the surface area of the simulation cell, i and j are the initial and final band respectively and s is the spin channel. For the one step model the amount of factors is slightly reduced and thus the QE is calculated as

$$QE(\omega, \mathbf{k}, \mu, T, s) = \frac{1}{A} \sum_{i,j,s} \Gamma_{i \rightarrow j} W(i, j, \mathbf{k}, \mu, s) n(i, T, s) I(\omega, \mu) \\ esc(\omega, f, \mathbf{k}, \mu, s) \Theta(\mathbf{k}_\perp) (1 + D(i, s)) \quad (2.43)$$

The atomic weighting of the contributions of the atomic sites allows the projected quantum efficiency to be defined as;

$$QE(\omega, \mu, T) = \int_{BZ} QE(\omega, \mathbf{k}, \mu, T) \frac{d\mathbf{k}}{8\pi^3} \quad (2.44)$$

The accuracy of the computed surface photoemission can be limited by the computational resources. First principle simulations of surfaces are typically performed in using the slab model. In this model, a 2D periodic slab of material of finite thickness in the third dimension (a periodic "slab") separated from the repetitive images by a vacuum region is used to model a semi-infinite surface. The properties of the surface simulated using the slab model usually converges quickly with respect to the slab thickness, so only a small, finite number of atomic layers is required. Sun and Ceder developed an efficient scheme for the creation and convergence of surface slabs that was able to converge the surface energy with slabs as thin as seven atomic layers. However, photoemitted electrons can be emitted from layers deeper inside the material, in the orders of nanometres (tens to hundreds of atomic layers).

Since the electronic properties of the slab in a suitable construction of a surface converges with respect to the slab thickness for a small finite number of atomic layers, the centre of the slab approximates bulk conditions. It is therefore possible to compute the photoemission efficiently from layers many nanometres from the surface by replicating bulk layers to simulate the photoemission from a slab model of arbitrary thickness. In the current implementation, the contribution of the deeper atomic layers to the photoemission process is approximated by the repetition of the atomic layer at the centre of the slab. The QE contribution of the bulk states is expressed as

$$QE_{bulk}(\omega, T) = \sum_{\mu=N}^{\tau} QE(\omega, \mu, T) \quad (2.45)$$

where N is the index of the atomic layer at the centre of the slab and

$$\tau = \frac{n_{IMFP} \lambda}{d(N)} \quad (2.46)$$

defines the maximum number of layers to consider as a multiple of the IMFP value supplied by the user. Here λ is the IMFP value, n_{IMFP} is an integer to determine the cutoff distance for the bulk slab approximation as a multiple of λ (default = 10), and $d(N)$ is the thickness of the slab layer.

The total quantum efficiency $QE_{tot}(\omega, T)$ is the sum of the explicitly computed N surface layers and the bulk contribution

$$QE_{tot}(\omega, T) = \sum_{\mu}^N QE(\omega, \mu, T) + QE_{bulk}(\omega, T) \quad (2.47)$$

The mean transverse energy

$$MTE(\omega, T) = \int_{BZ} \frac{\sum_{\mu}^N QE(\omega, \mathbf{k}, \mu, T) \frac{\hbar^2}{m_e} \mathbf{k}_{\parallel}^2}{QE_{tot}(\omega, T)} \frac{d\mathbf{k}}{8\pi^3} \quad (2.48)$$

is the sum of the QE contribution at each atomic site times the transverse energy of a particular $\mathbf{k}$ point, normalized to the total quantum efficiency integrated over the Brillouin zone (BZ).

This could also be described as a weighted sum for the contributions across all the $\mathbf{k}$ points.

2.5.10 Transverse momentum for supercell geometries

When using $\mathbf{k}$ points to calculate the electron's transverse momentum a problem arises when calculating the electronic structure of supercell (SC) structures. With a SC, a phenomenon generally called "band folding" occurs. This means, that the bands start being "folded" towards the Γ point at the center of the BZ. More specifically a band present at the PC zone boundary is "folded" onto the Γ point. If one now calculates the transverse momentum associated with that band using just the $\mathbf{k}$ point's one gets unsurprisingly 0, rather than $\pi/\mathbf{a}$. Since that transverse momentum has an important role in deciding if an emission is allowed, this means, that this specific emission is now possible at much lower photon energies. In addition the transverse energy distribution is also reduced, here to 0 specifically. These two differences are present for all "folded" bands and in total show the effect of a much higher QE value and lower MTE value when comparing the PC and SC geometries.

One can, however, recover the original transverse momentum. For this each of the $\mathbf{k}$ points is expanded into a grid of $\mathbf{k}_{\mathbf{x},\mathbf{y}} + \mathbf{G}_{\mathbf{x},\mathbf{y}}$ points, where $\mathbf{G}_{\mathbf{x},\mathbf{y}}$ are the reciprocal cell's x and y vectors. The grid's coordinates in $1/\text{\AA}$ are calculated by

$$c_x(\mathbf{k}_x + \mathbf{G}_x) = \mathbf{k}_x + \alpha \cdot \mathbf{G}_x(\mathbf{x}) + \beta \cdot \mathbf{G}_y(\mathbf{x}) \quad (2.49)$$

$$c_y(\mathbf{k}_y + \mathbf{G}_y) = \mathbf{k}_y + \alpha \cdot \mathbf{G}_x(\mathbf{y}) + \beta \cdot \mathbf{G}_y(\mathbf{y}) \quad (2.50)$$

where $\mathbf{k}$ is in cartesian coordinates and

$$\begin{aligned} [\alpha] &= \{x \in \mathbb{Z} \mid -\alpha_{max} \leq x \leq \alpha_{max}\} \\ [\beta] &= \{x \in \mathbb{Z} \mid -\beta_{max} \leq x \leq \beta_{max}\}. \end{aligned}$$

The values of α_{max} and β_{max} are chosen, so that $|c_x| < 3.8 \text{ } 1/\text{\AA}$ and $|c_y| < 3.8 \text{ } 1/\text{\AA}$ to limit the number of grid points. The value on each of the grid points is calculated by summing up the u_f coefficients for the z direction as

$$w_{gk}(\mathbf{k}_{\mathbf{x},\mathbf{y}} + \mathbf{G}_{\mathbf{x},\mathbf{y}}) = \sum_{G_z} |u_f(\mathbf{G}, \mathbf{k}_{\mathbf{x},\mathbf{y}})|^2 \quad (2.51)$$

where u_f are the plane wave basis coefficients. The $c_{x,y}$ and w_{gk} values are read from a `*.gkgrid_bin` file. The calculation of QE and MTE values is performed analogous to Section 2.5.9 for each of the grid points with $c_{x,y}$ replacing $\mathbf{k}_{||}$.

Chapter 3

Getting Started

3.1 Installation

OPTADOS is usually obtained in a gzipped tarball, `optados-X.X.tar.gz`. Extract this (`tar -xzf optados-X.X.tar.gz`) in the desired directory. Inside the `optados/` directory are a number of sub directories, `documents/`, `examples/` and `tools/`. The code may be compiled using the Makefile in the `optados/` directory. The `SYSTEM`, `BUILD`, `COMMS_ARCH` and `PREFIX` flags must be set, either in the `makefile.system`, or from the command line (for example `make BUILD=fast`).

3.1.1 SYSTEM

Choose which compiler to use to make OPTADOS. The valid values are:

- `gfortran` (default)
- `ifort`
- `nag`
- `pgf90`
- `A00C`
- `oneAPI`

The following are no longer supported but are provided for legacy systems:

- `g95`
- `pathscale`
- `sun`

3.1.2 BUILD

Choose the level of optimisations required when making OPTADOS. The valid values are:

- `fast` (default) All optimisations
- `debug` No optimisations, full debug information

3.1.3 COMMS_ARCH

Whether to compile for serial or parallel execution. The valid values are:

- `serial` (default)
- `mpi`

3.1.4 PREFIX

Choose where to place the OPTADOS binary. The default is the OPTADOS directory.

3.2 Usage

`optados.x86_64 [seedname]`

- `seedname`: If a seedname string is given the code will read its input from a file `seedname.odi`. The default value is `CASTEP`.

Chapter 4

Structure of the Program

The schematic structure of the program is outlined in Fig. 4.1. Each rounded-cornered box represents a Fortran95 module and the lines represent module dependencies. Modules only use data and subroutines from lower modules in the diagram. The modules are grouped into functional, structural, utility and low level. Low level modules set up the environment for communication in serial or parallel between nodes and to disk. The utility modules are not context dependent on the problem, and contain general algorithms and an interface to write `xmgrace` input files. Structural modules define data structures, hold data and perform low-level operations on them. The functional modules perform the higher-level data operations and generate output.

A description of the purpose of each module is given below.

- `algorithms`: subroutines not specific to electronic structure
- `cell`: real and reciprocal space data manipulation and storage, (*e.g.* cell vectors and k -points)
- `comms`: interfaces to parallel libraries
- `constants`: definition of constants and conversion factors
- `core`: perform core-level calculation
- `dos`: perform DOS calculation
- `dos_utils`: perform type (a) integral evaluation
- `electronic`: read electronic eigenvalue data. Store electronic data variables
- `io`: error handling, timing, and input and output units
- `jdos`: perform JDOS calculation
- `jdos_utils`: perform type (b) integral evaluation
- `optados`: main program

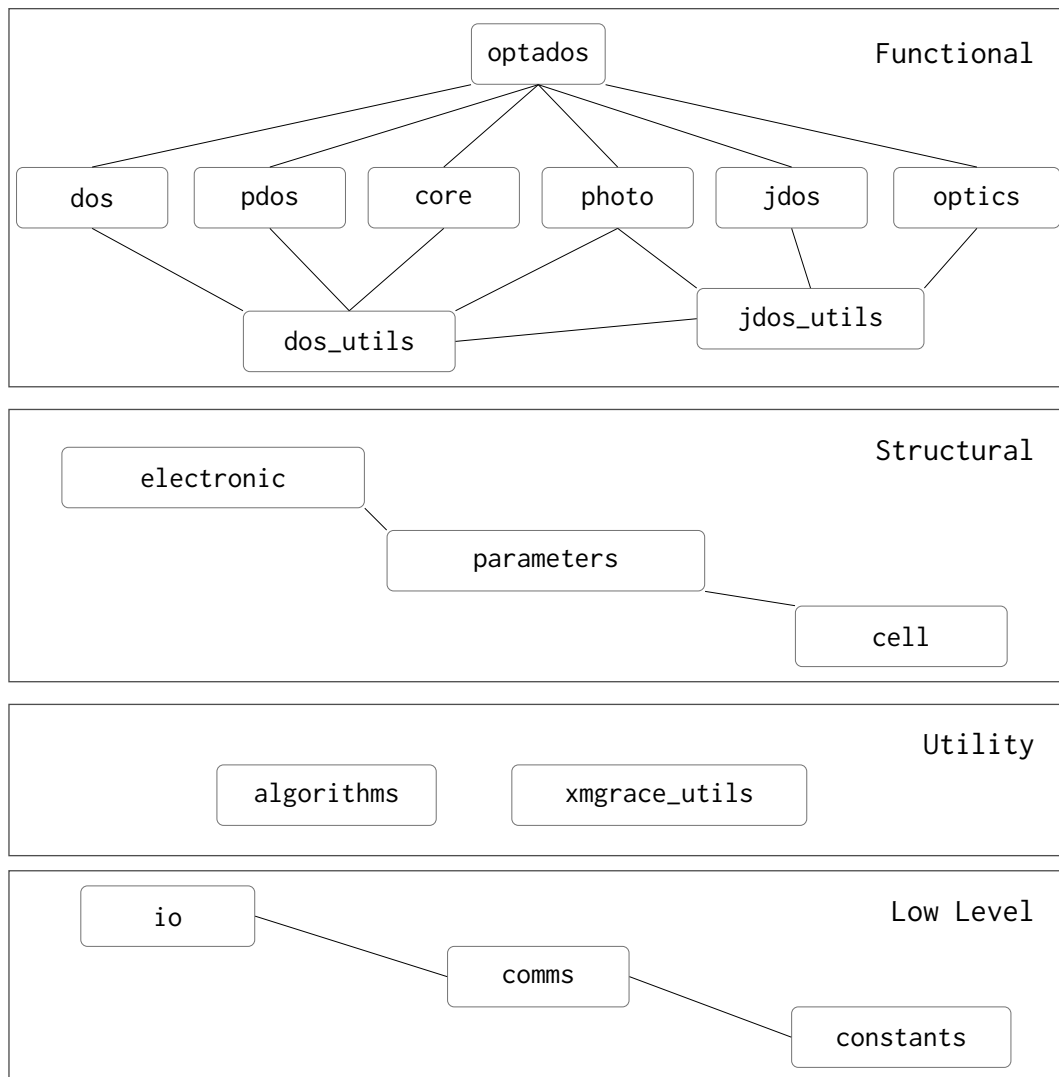


Figure 4.1: Schematic structure of the program

- **optics**: calculate dielectric function then generate optical properties
- **parameters**: read, store and check input file parameters
- **photoemission**: perform a photoemission characteristics calculation
- **pdos**: perform PDOS calculation
- **pdis**: perform a projected dispersion calculation
- **projection_utils**: routines for reading and writing projectors
- **xmgrace_utils**: routines to write out data in xmgr format

4.1 Structure of the Photoemission module

- **analyse_geometry**: this subroutine identifies which atoms belong to each layer and which is the maximum one to include in the calculations (half slab). It contains:
 - a sorting algorithm to get an ordering by the atom's z coordinate
 - an algorithm to calculate the center of the slab and define a set of equally spaced boxes representing layers within the slab
 - identification of the maximum layer to use for the calculation
 - identification of the maximum atom to use for the calculation
 - identification of the number of atoms in each of the layers
- **calc_band_info**: this subroutine checks that the supplied band energies are sorted in ascending order, as that is essential in the later stages of the calculation. The included steps are:
 - check that the band energies are in ascending order at every k-point
 - save the index for the first unoccupied band at every k-point
- **calc_photon_energies**: this subroutine allows the photoemission module to calculate the photoemission and other characteristics for a number of photon energies in a single run. It calculates the number of steps to take during the run and sets all the necessary variables to properly initialise arrays later.
- **make_pdos_weights_atoms**: this subroutine calculates the PDOS on atoms by summing the orbital contributions for a certain atom. It is taken from the *pdos_merge* subroutine of *pdos.F90*, but only the sum over atoms is included. It contains:
 - the sum of the orbital contribution of the same atom
 - the sum of all atomic contribution at fixed **k** and band indices.
- **calc_photo_optics**: this subroutine calculates the optical properties of the system using the following steps:

- calculate the optical weights using **od_optics : make_weights** For the photoemission calculation, the `optics_geom` can be
 - * **polarized**: the three indices specify the vector of oscillation for the incoming radiation's electric field;
 - * **unpolarized**: the three indices define a plane perpendicular to the direction of the light propagation. See Section 5.7 for further details.

Full dielectric tensor calculation is not implemented for photoemission calculations.

- weight the *optical weights* using the `pdos_weights_atoms`
 - calculate the atom projected joint density of states (jDOS) for each of the atoms using **jdos_utils_calculate**
 - calculate the imaginary part of the dielectric function ϵ_2 using **od_optics : calc_epsilon_2**
 - calculate the real part of the dielectric function ϵ_1 with the Kramers-Kronig relations using **od_optics : calc_epsilon_1**
 - calculate the refractive index n of each atom using **od_optics : calc_refract**
 - calculate the absorption coefficient A of each atom using **od_optics : calc_absorp**
 - calculate the reflectivity of each atom using **od_optics : calc_reflect**
- **calc_absorp_layer**: this subroutine calculates the portion of light absorbed by each layer. If there is only one layer, the thickness is fixed to one.
 - **effective_wf**: this subroutine calculates the reduction of the work function in line with the Nordheim-Fowler theory when applying an electric field.[15]
 - **calc_field_emission**: this subroutine uses Nordheim-Fowler barrier theory, to calculate the probability of an electron in a band with a certain energy to tunnel through the reduced barrier height, see 2.5.8 for the equations
 - **calc_angle**: this subroutine calculates the electron's the transverse energy and its photoemission angles θ/ϕ within the slab and after passing into the vacuum.
 - **calc_electron_esc**: this subroutine calculates the escape length and the probability for the emission of the electron from a each of the layers. The probability is calculated according to the inelastic mean free path (IMFP) theory. The user can supply either a **single IMFP value**, that will be used for the entire material, or a **list of IMFP values** for each layer being calculated. In the second case, a weighted IMFP value is calculated for each layer based on the thickness of each layer and its respective location within the slab.
 - **bulk_emission**: Due to computational constraints, the thickness of a slab model cannot be made very high. To account for the lack of bulk like material, that can still contribute to the overall photoemission, the most internal layer is replicated until emitted electrons have a chance of reaching the surface of $< 1 \cdot 10^{-5}$. The contributions from these layers are summed up.
 - **QE calculation**: Calculates the layer-by-layer QE. Two options are available:

- **calc_three_step_model**: Calculates the QE assuming a final Bloch state (conduction band) in the crystal.
- **calc_one_step_model**: Calculates the QE assuming a final free electron state.
- **weighted_mean_te**: Weights the contribution to the transverse energy of each individual state at a certain k-point according to its QE contribution.
- **write_qe_data**: this subroutine takes the calculated contributions and MTE and writes it to the output file concisely for the user.
- **extra calculations**: A number of options are available.
 - **qe_tensor**: Write out the contributions of each band for all k-points, spins and atoms. This can be used to study the specific origin of photoemitted electrons, for detailed visualisations and much more.
 - **bindenergy_curve**: Calculate and write out a **QE vs binding energy** plot. A Gaussian broadening is applied. Additionally, the ranges for emission angles θ and ϕ can be set to allow for comparison with ARPES data, *e.g.* energy distribution curves (EDC).
 - **binding_energy_momentum_map**: Calculate and write a **binding energy vs transverse energy** map as a matrix with the value of each grid point representing the QE contribution at that point. The contributions are Gaussian broadened and the ranges for emission angles θ and ϕ can be set to allow for comparison with ARPES data.
 - **const_binding_energy_map**: Calculate and write a p_x vs p_y matrix of grid points where the value at each grid point represents the total QE contribution at a specified binding energy. The specified binding energy is w.r.t. E_F .
 - **p_tensor**: Calculate and write a p_x, p_y, p_z tensor of grid points for electron momenta in $\frac{1}{\text{\AA}}$. Each value at that grid point represents the QE contribution at that specific set of electron momenta. The contributions are Gaussian broadened and the ranges for emission angles θ and ϕ can be set to allow for comparison with ARPES data.

Chapter 5

Parameters

5.1 seedname.odi File

The OPTADOS input file `seedname.odi` has a flexible free-form structure.

The ordering of the keywords is not significant. Case is ignored (so `smearing_width` is the same as `Smearing_Width`). Characters after `!`, or `#` are treated as comments. Most keywords have a default value that is used unless the keyword is given in `seedname.odi`. Keywords may be set in any of the following ways

```
smearing_width = 0.4
smearing_width : 0.4
smearing_width 0.4
```

A logical keyword can be set to `.true.` using any of the following strings: `T`, `true`, `.true..`

5.2 General Parameters

5.2.1 `character(len=20) :: task`

Tells the code what to compute.

The valid options for this parameter are:

- `dos` (default)
- `compare_dos`
- `compare_jdos`
- `jdos`
- `pdos`
- `pdispersion`
- `optics`
- `core`
- `photoemission`
- `photo_energy_sweep`
- `all`

Several tasks can be specified *e.g.* to compute `dos` and `jdos` use `task : dos jdos`. However, the `compare_dos` and `compare_jdos` tasks can only be combined with each other, and no additional tasks. `compare_dos` and `compare_jdos` calculate the DOS and JDOS respectively using all broadening schemes. It is good practice to check the quality of the underlying DOS before other tasks are requested. When setting the task to `photoemission`, a photoemission calculation for a single photon energy is performed. By choosing `photo_energy_sweep` as range of photon energies can be defined to do a series of photoemission calculations at different photon energies.

5.2.2 `character(len=50) :: broadening`

Specified the scheme used to broaden a discrete sampling of the Brillouin Zone to a continuous spectral function.

The valid options for this parameter are:

- `adaptive` (default)
- `fixed`
- `linear`
- `quad` (not currently implemented)

5.2.3 integer :: iprint

This indicates the level of verbosity of the output from 1, the bare minimum: 2, with progress reports: to 3, which corresponds to full debugging output.

The default value is 1.

5.2.4 character(len=20) :: energy_unit

The energy unit to be used for writing quantities in the output files.

The valid options for this parameter are:

- eV (default)
- Ry
- Ha

5.2.5 logical :: legacy_file_format

- TRUE Read CASTEP input compatible with versions < 6.0.
- FALSE (Default) Read CASTEP input compatible for use with CASTEP versions 6.0+ and generated with the castep spectral task.

5.2.6 real(kind=dp) :: adaptive_smearing

Set the relative smearing in the adaptive scheme.

Default value is 0.4

5.2.7 real(kind=dp) :: fixed_smearing

Smearing width for fixed broadening.

If `spectral_scheme = fixed` default value is 0.3eV.

5.2.8 real(kind=dp) :: linear_smearing

Smear the linear broadening with a Gaussian of this width.

Default value is 0.0.

5.2.9 `character(len=20) :: efermi`

Choose which Fermi energy to use.

The valid options for this parameter are:

- `optados` (default) OPTADOS recalculates the Fermi energy by performing a DOS calculation.
- `file` Take the value from the output of the ab-initio calculation.
- `insulator` Assume that the material is an insulator and counts filled bands to find the Fermi energy.
- `<real number>` User supplied value.

The default value is `optados`.

5.2.10 `character(len=20) :: output_format`

Format in which to output data.

The valid options for this parameter are:

- `gnuplot`
- `grace` (default)

5.2.11 `logical :: finite_bin_correction`

Force each Gaussian to be larger than a single energy bin. (Useful for adaptive smearing and semi-core states when `numerical_intdos=TRUE`).

Default value `TRUE`.

5.2.12 `logical :: numerical_intdos`

Calculate the integrated dos by numerical integration instead of semi-analytically. (Useful for comparison with `LinDOS`.)

Default value `FALSE`.

5.2.13 `logical :: hybrid_linear`

Switch from linear broadening scheme to adaptive broadening when band gradient less than `hybrid_linear_grad_tol`. This allows for a good description of very flat bands such as defect

and semi-core states. May also be used in conjunction with `finite_bin_correction` further improving the DOS and band energy

Default value FALSE.

5.2.14 `real(kind=dp) :: hybrid_linear_grad_tol`

Tolerance for switching from linear to adaptive broadening when using `hybrid_linear` option.

The default value is 0.01eV/Å.

5.2.15 `character(len=50) :: devel_flag`

Not a regular keyword. Its purpose is to allow a developer to pass a string into the code to be used inside a new routine as it is developed.

No default.

5.3 DOS Parameters

5.3.1 `logical :: compute_band_energy`

Compute the band energy by summing bands both using CASTEP's eigenvalue and OPTADOS's density of states.

Default value TRUE.

5.3.2 `logical :: compute_band_gap`

Compute the optical, thermal and average band gap.

Default value FALSE.

5.3.3 `logical :: dos_per_volume`

Present DOS per simulation cell volume.

Default value FALSE.

5.3.4 `real(kind=dp) :: dos_min_energy`

Lower energy range for DOS and related properties.

Default value is 5eV below the lowest eigenvalue in the bands file.

5.3.5 `real(kind=dp) :: dos_max_energy`

Upper energy range for DOS and related properties.

Default value is 5eV above the highest eigenvalue in the bands file.

5.3.6 `real(kind=dp) :: dos_nbins`

Instead of setting a Default value `dos_spacing` the total number of DOS bins may be given. (Useful for comparison with LinDOS.)

5.3.7 `real(kind=dp) :: dos_spacing`

Resolution at which to compute the DOS and related properties.

Default value is 0.1eV

5.3.8 `logical :: set_efermi_zero`

Shift energy scales so that the Fermi energy is at 0.

Default value FALSE.

5.4 JDOS Parameters

5.4.1 `real(kind=dp) :: jdos_max_energy`

Upper energy range for JDOS and related properties.

Default value is the difference between the valence band maximum (or Fermi level) and the highest eigenvalue in the bands file.

5.4.2 `real(kind=dp) :: jdos_spacing`

Resolution at which to compute the DOS and related properties.

Default value is 0.01eV.

5.4.3 `real(kind=dp) :: scissor_op`

Value of the scissor operator.

Default value is 0 eV (*i.e.* not used)

5.4.4 `real(kind=dp) :: exclude_bands`

This allows a list of bands which are NOT to be included in the JDOS or OPTICS calculation to be specified. The bands 1, 2, 4, 5, 6, for example, can be specified using `exclude_bands = 1,2,4-6`.

5.5 PDOS Parameters

5.5.1 `character :: pdos`

Defines which components to include in the pdos analysis:

- `angular` (decompose as s,p,d *etc.*)
- `species` (decompose onto atomic species C, H *etc.*)
- `sites` (decompose onto atomic sites C1, H1, H2 *etc.*)
- `species_ang` (decompose onto angular momentum channels and species Cs, Cp *etc.*)
- `C:H` (decompose onto Carbon and Hydrogen sites)
- `C1:C3:C4-C8` (decompose onto atoms C1, C2 and C4,C5,C6,C7,C8)
- `Si1(s;d)` (decompose onto 's' and 'd' channels for atom Si1)
- `sum:C1:C3:C4-C8` (decompose onto atoms C1, C2 and C4,C5,C6,C7,C8 and combine into the single projection)
- ```C:Exi``:H` (decompose only onto site labelled C:Exi in `seedname-out.cell` and Hydrogen sites)
- ```C``:H` (decompose only onto Carbon sites that *are not labelled* in `seedname-out.cell` and Hydrogen sites. *N.B* this is the same as `C:H` if no sites are labelled.)

5.6 Projected Dispersion Parameters

5.6.1 `character :: pdispersion`

Follows `pdos` in specifying which projectors are included in the projected dispersion calculation. Note that this task is not compatible with any options other than `EFERMI: FILE` (which is the default when `pdispersion` has been requested).

5.7 Optics Parameters

5.7.1 `character(len=20) :: optics_geom`

Specifies the geometry for the optics calculation. Possible options:

- `polycrystalline` (Isotropic average)
- `polarized`
- `unpolarized`
- `tensor` (Full dielectric tensor)

The default is `polycrystalline`.

5.7.2 `real(kind=dp) :: optics_qdir(3)`

Direction of polarisation. Must be specified if `optics_geom = polarized` or `optics_geom = unpolarized`. There is no default value

5.7.3 `logical :: optics_intraband`

If true, the intraband contribution to the dielectric function will be calculated. (Important for metals.)

The default is `FALSE`.

5.7.4 `real(kind=dp) :: optics_drude_broadening`

Value of broadening included in the Drude term expressed in s^{-1} .

The default value is `1E-14`.

5.7.5 `real(kind=dp) :: optics_lossfn_broadening`

FWHM of Gaussian used to broaden the loss function.

The default value is `0` (*i.e.* no broadening is used).

5.8 Core-level Parameters

5.8.1 `character(len=20) :: core_type`

Determines if we want absorption (transition from core to conduction ELNES / XANES) or emission (transition from valence to core XES). It is also possible to plot both.

- absorption (default)
- emission
- all

5.8.2 `character(len=20) :: core_geom`

Specifies the geometry for the core-spectra calculation. Possible options:

- polycrystalline (Isotropic average)
- polarized

The default is polycrystalline.

5.8.3 `real(kind=dp) :: core_qdir(3)`

Direction of polarisation. Must be specified if `core_geom = polarized`.

This should be specified as three real numbers, *e.g.* 1 0 0.

There is no default value.

5.8.4 `logical :: core_LAI_broadening`

Include life-time and instrumentation broadening.

The default is FALSE.

5.8.5 `real(kind=dp) :: LAI_gaussian_width`

FWHM of Gaussian function used to broaden spectrum.

The default value, if `core_LAI_broadening = true`, is 0 (*i.e.* no Gaussian used).

5.8.6 `real(kind=dp) :: LAI_lorentzian_width`

FWHM of fixed Lorentzian function used to broaden spectrum.

The default value, if `core_LAI_broadening = true`, is 0 (*i.e.* no fixed Lorentzian used).

5.8.7 `real(kind=dp) :: LAI_lorentzian_scale`

Variation of Lorentzian function with energy *i.e.* the width of the Lorentzian is (energy above the Fermi level x `core_lorentzian_scale`). If set to zero, no energy dependent broadening is included. If `core_lorentzian_scale` and `core_lorentzian_width` are both specified, the total width of the Lorentzian used will be `core_lorentzian_width` + (energy above the Fermi level x `core_lorentzian_scale`).

The default value, if `core_LAI_broadening = true`, is 0.1.

5.8.8 `real(kind=dp) :: LAI_lorentzian_offset`

Energy (in eV) above the Fermi level that the energy dependent broadening starts.

The default value, if `core_LAI_broadening = true`, is 0.

5.9 Photoemission Parameters

5.9.1 `character(len=20) :: photo_model`

Set the final state for the photoemitted electrons.

- `3step` (default)
Read the matrix elements and calculate the photoemission assuming a conduction band final state in the solid.
- `1step`
Read the matrix elements and calculate the photoemission assuming a free electron final state.
- `dosds`
Calculate a DOS dependent emission model based on an extension of the Dowell-Schmerge Model, calculates both individual band contributions and full DOS contributions. Adapted from the model published by Saha, et al.. [20]

5.9.2 `character(len=20) :: photo_momentum`

Set the way the photoelectron's transverse momentum to the surface is computed.

- **crystal** (default)
Takes the transverse crystal momentum $\mathbf{k}_{x,y}$ to compute the electrons transverse energy.
- **gkgrid**
Instead of using a set of crystal momentum $\mathbf{k}$ values on a grid, *e.g.* Monkhorst Pack (MP), each $\mathbf{k}$ point is expanded into a set of grid points. This grid is centered on each of the crystal momenta $\mathbf{k}_{x,y}$ and expanded using the reciprocal lattice vectors $\mathbf{G}_x$ and $\mathbf{G}_y$. This option allows to get consistent results for both primitive cell (PC) and super cell (SC) slab geometries. When using **crystal**, the QE/MTE values for SC slabs are over/underestimated due to the bands being folded towards the Γ point in SC geometries.

5.9.3 `real(kind=dp) :: photo_photon_energy`

Sets the photon energy at which the photoemission characteristics are calculated. If `task = photoemission`, `photo_photon_energy` has to be set. If `task = photo_energy_sweep`, this value **must not** be set, otherwise OptaDOS will give an error message and terminate.

There is no default value

5.9.4 `real(kind=dp) :: photo_photon_min`

If `task = photo_energy_sweep` this represents the initial energy of the photon energy sweep. The photon energy will be increased for each step by the value of `jdosing_spacing`.

5.9.5 `real(kind=dp) :: photo_photon_max`

If `task = photo_energy_sweep` this represents the final energy of the photon energy sweep. The photon energy will be increased for each step by the value of `jdosing_spacing` up to and including `photo_photon_max`.

ATTENTION : If setting `photo_photon_min/max` the difference between them has to be an integer multiple (including 0) of `jdosing_spacing`, otherwise OptaDOS will give an error message and terminate.

5.9.6 `real(kind=dp) :: photo_slab_min`

Sets the assumed lower boundary of the material's slab in case only the upper and lower limit are given. This value is used to calculate the slab's volume and center. It should be the same distance from the assumed center of the material slab to ensure the slab's center can be reliably found by OptaDOS. This needs to be determined/estimated by the user and could for example be the coordinate of the atom with the lowest z-coordinate plus that atom's vdW-radius.

There is no default value

5.9.7 `real(kind=dp) :: photo_slab_max`

Sets the assumed upper boundary of the material's slab in case only the upper and lower limit are given. This value is used to calculate the slab's volume and center. It should be the same distance from the assumed center of the material slab to ensure the slab's center can be reliably found by OptaDOS. This needs to be determined/estimated by the user and could for example be the coordinate of the atom with the highest z-coordinate plus that atom's vdW-radius.

There is no default value

5.9.8 `real(kind=dp) :: photo_slab_middle`

Sets the assumed middle of the slab in the case that the layer boundaries are determined by the user input. When this is set, `photo_slab_min` and `photo_slab_max` **cannot** be set and `photo_layers_tops` **must** be set. This value is used to calculate the height of the innermost layer with the last value in the vector of `photo_layers_tops`.

There is no default value

5.9.9 `real(kind=dp) :: photo_layers_tops(#layers)`

This vector sets the assumed top z-coordinate for the layers going from the surface towards the middle. Split each value by a space. The first value is inferred to be the slab boundary and each layer that the user wants to include in the calculation needs to be represented in the vector. This makes it possible to calculate the layer heights, layer volumes and atoms in a layer. Some more user input is needed than if `photo_slab_min` and `photo_slab_max` were set, but there are way less assumptions by the program.

There is no default value

5.9.10 `real(kind=dp) :: photo_work_function`

Sets the assumed work function for the calculation. This needs to be determined/estimated by the user.

There is no default value

5.9.11 `real(kind=dp) :: photo_bulk_cutoff`

Sets the multiple of the IMFPs after which to terminate the bulk approximation. The number of layers in the bulk approximation depends on the value(s) set in `photo_imfp_value`.

The default value is 10.0

5.9.12 `real(kind=dp) :: photo_elec_field`

Sets the external electric field in V/m.

The default value is 0.0 V/m

5.9.13 `character(len=20) :: photo_imfp_model`

- `const` (default)
- `layers`
- `cu_curve` (experimental)

This sets the model for how to treat the `photo_imfp_value` input. For the `const` option a single value is used for all layers in the slab. For the `layers` option each explicit layer must be given a value. The IMFP value for a specific layer is calculated as a weighted average of all the above layers. The `curve` option is only experimental and calculates an IMFP value based on each electron's excess energy during propagation within the material. The model for the curve was parametrized for Cu and published by Nagy and Echenique. [14]

5.9.14 `real(kind=dp) :: photo_imfp_value(1 or #layers)`

Set the inelastic mean free path (IMFP) value(s) in Å for the calculation. See `photo_imfp_model` for further explanation. When using `layers` the number of values must equal the number of explicit layers being used for the calculation. This is generally $\# \text{ slab layers}/2$ for an even number of total layers and $\# \text{ slab layers}/2 + 1$ for an odd number of total layers. In case the user gives the layer boundaries, the number of layers determined, has to match the number of layers given in `photo_layers_tops`.

There is no default value and the parameter must be set for all calculations.

5.9.15 `real(kind=dp) :: photo_temperature`

Sets the assumed temperature during the calculation in Kelvin. The occupation of bands is smeared using a Fermi-Dirac distribution with this temperature value.

The default value is 298 K

5.9.16 `character(len=80) :: photo_output`

- `off` (default)

No extra output is written.

– `qe_tensor`

Write out the contributions of each band for all $\mathbf{k}$ -points, spins and atoms. This can be used to study the specific origin of photoemitted electrons, for detailed visualisations and much more.

– `bindenergy_curve`

Calculate and write out a QE vs *binding energy* curve. A Gaussian broadening of width `photo_bindenergy_broadening` is applied. The energy bin width = 0.001 eV. The considered emission angle ranges are limited by the values for the min/max values θ and ϕ set to allow for comparison with ARPES data, *e.g.* energy distribution curves (EDC).

– `ekin_ptrans_map`

Calculate and write a *kinetic energy* vs *transverse momentum* map as a matrix with the value of each grid point representing the QE contribution at that point. The energy bin width = 0.001 eV. The transverse momentum bin width is controlled by `photo_kmat_bin_width`.

A Gaussian broadening of width `photo_bindenergy_broadening` is applied. The broadening in $\mathbf{p}$ has a FWHM of $1/2$ distance between $\mathbf{k}$ points, *i.e.* a half step between MP grid points. The emission angle ranges are limited by the values for the min/max values θ and ϕ set to allow for comparison with ARPES data.

– `const_bindenergy_p_map`

Calculate and write a $\mathbf{p}_x$ vs $\mathbf{p}_y$ matrix of grid points where the value at each grid point represents the total QE contribution at a constant binding energy across the entire map. The momentum bin width is controlled by `photo_kmat_bin_width`. The binding energy value is set by `photo_const_bindenergy_value` w.r.t. the Fermi energy (E_F).

The contributions are Gaussian broadened in energy with a width of `photo_bindenergy_broadening`. The broadening in the $\mathbf{p}_x$ vs $\mathbf{p}_y$ direction has a FWHM of $1/2$ distance between $\mathbf{k}$ points, *i.e.* a half step between MP grid points. The emission angle ranges are limited by the values for the min/max values θ and ϕ set to allow for comparison with ARPES data.

– `p_tensor`

Calculate and write a $\mathbf{p}_x, \mathbf{p}_y, \mathbf{p}_z$ tensor of grid points for electron momenta in $1/\text{\AA}$. Each value at that grid point represents the QE contribution at that specific set of electron momenta values.

The contributions are Gaussian broadened in $\mathbf{p}_x$ and $\mathbf{p}_y$ with a FWHM of half a step between $\mathbf{k}$ grid points, *i.e.* MP grid points. In the $\mathbf{p}_z$ direction the value `photo_bindenergy_broadening` is converted to $1/\text{\AA}$ to broaden in the $\mathbf{p}_z$ direction. The ranges for emission angles θ and ϕ can be set to allow for comparison with ARPES data.

5.9.17 `real(kind=dp) :: photo_theta_min`

Sets the lower limit of the polar emission angle θ for contributions when calculating angle dependent quantities. See `photo_output` for examples.

The default value is 0°

5.9.18 `real(kind=dp) :: photo_theta_max`

Sets the lower limit of the polar emission angle θ for contributions when calculating angle dependent quantities. See `photo_output` for examples.

The default value is 90°

5.9.19 `real(kind=dp) :: photo_phi_min`

Sets the lower limit of the azimuthal emission angle ϕ for contributions when calculating angle dependent quantities. See `photo_output` for examples.

The default value is 0°

5.9.20 `real(kind=dp) :: photo_phi_max`

Sets the lower limit of the azimuthal emission angle ϕ for contributions when calculating angle dependent quantities. See `photo_output` for examples.

The default value is 90°

5.9.21 `real(kind=dp) :: photo_bindenergy_broadening`

Sets the Gaussian broadening width in the energy dimension for `photo_output`.

The default value is $0.0257 \text{ eV} = k_b \cdot 298 \text{ K}$

5.9.22 `real(kind=dp) :: photo_pmat_bin_width`

Sets the bin width in the momentum dimension for `photo_output`.

The default value is $0.005 \text{ 1/\AA}$

5.9.23 `real(kind=dp) :: photo_const_bindenergy_value`

Sets the binding energy value for the `photo_output` option `const_bindenergy_p_map` w.r.t the Fermi energy.

The default value is 0.0 eV, *i.e.* E_F

5.9.24 `logical :: photo_use_tmprob`

Defines if the probability for transmission across the surface step should be included when calculating the three step model.

The default value is `True`. This requires a `SEEDNAME.tmprob_bin` file to be present.

Chapter 6

CASTEP Examples

Each example in the `examples/` directory contains example CASTEP input files and a sample OPTADOS input file. To keep the OPTADOS distribution light CASTEP output files for the examples have not been provided. You need to run CASTEP on the CASTEP input files before running OPTADOS on the examples.

6.1 Density of States

- *Outline* : This is a simple example of using OPTADOS for calculating electronic density of states of crystalline silicon in a 2 atom cell.
 - *Input Files*:
 - `examples/Si2_DOS/Si2_dos.cell` - The CASTEP cell file containing information about the simulation cell.
 - `examples/Si2_DOS/Si2_dos.param` - The CASTEP param file containing information about the parameters for the SCF and spectral calculations.
 - `examples/Si2_DOS/Si2_dos.odi` - The OPTADOS input file, containing the parameters necessary to run OPTADOS.
1. Perform a CASTEP calculation on the bulk silicon using the `Si2_dos.cell` and `Si2_dos.param` input files.

```
$ castep Si2
```

This should take a couple of seconds to run. More help can be found in the tutorials on the CASTEP website www.castep.org.
 2. Perform an OPTADOS calculation. Add `LEGACY_FILE_FORMAT : true` in the `Si2_DOS.odi` input file, if the CASTEP version you are using is before 6.0. Then execute:

```
$ optados.<SYSTEM>.<BUILD>.<COMMS_ARCH>.x86_64 Si2
```

This generates 3 files:

- `Si2.odo` – OPTADOS general output file.
 - `Si2.adaptive.dat` – The adaptive broadened DOS raw output data.
 - `Si2.adaptive.agr` – The adaptive broadened DOS in a file suitable to be plotted by `xmgrace`.
3. Open the `Si2.odo` file in a text editor (*e.g.* `vi` or `emacs`). OPTADOS has performed a Density of States calculation.

```
+----- Fermi Energy Analysis -----+
| From Adaptive broadening                |
| Spin Component : 1 occupation between    3.99961 and    4.00003    <- 0c |
| Spin Component : 2 occupation between    3.99961 and    4.00003    <- 0c |
| Fermi energy (Adaptive broadening) :    5.4109 eV                <- EfA |
+-----+
```

It has used the integrated DOS to work out the Fermi level, and has suggested the error in the integration by indicating the number of electrons at the Fermi level. Since we had 4 up electrons and 4 down in the input file this analysis seems satisfactory.

```
+----- Electronic Data -----+
| Number of Bands                :          23                |
| Grid size                      :        10 x 10 x 10        |
| Number of K-points             :          110                |
| Spin-Polarised Calculation      :          True              |
| Number of up-spin electrons     :          4.00              |
| Number of down-spin electrons   :          4.00              |
+-----+
```

Since we had `efermi : optados`, OPTADOS sets the internal value of the Fermi level to the one it has derived from the DOS. This is important for subsequent calculations. Other valid options are `file`, where OPTADOS uses the value calculated by the electronic structure code that generated the eigenvalues; `insulator`, where OPTADOS uses a value calculated from assuming the system is non-metallic; or a value set by the user.

OPTADOS now performs some analysis of the DOS at the Fermi level,

```
+----- DOS at Fermi Energy Analysis -----+
| Fermi energy used :    5.4109 eV                |
| From Adaptive broadening                |
| Spin Component : 1  DOS at Fermi Energy :    0.0011 eIn/cell    <- DEA |
| Spin Component : 2  DOS at Fermi Energy :    0.0011 eIn/cell    <- DEA |
+-----+
```

From this we may assume that there is a band gap.

Importantly, then OPTADOS calculates the band energy from the DOS is has calculated.

```

+----- Band Energy Analysis -----+
|          Band energy (Adaptive broadening) :          1.3609 eV          <- BEA |
|          Band energy (From CASTEP) :          1.3622 eV          <- BEC |
+-----+

```

As the quality of the OPTADOS calculation is increased these two values should converge to the same answer.

Finally OPTADOS shifts the Fermi level to 0 eV, for the output files.

4. The DOS is outputted to `Si2.adaptive.dat`. This contains 5 columns as described in the header of the file:

```

#####
#
#               O p t a D O S   o u t p u t   f i l e
#
#   Density of States using adaptive broadening
#   Generated on 12 Feb 2012 at 16:50:37
# Column      Data
#   1          Energy (eV)
#   2          Up-spin DOS (electrons per eV)
#   3          Down-spin DOS (electrons per eV)
#   4          Up-spin Integrated DOS (electrons)
#   5          Down-spin Integrated DOS (electrons)
#
#####

```

This file can be plotted by your favourite graph-plotting software. However, OPTADOS has made things easy and generated a `Si2.adaptive.agr` file which is directly plottable using `xmgrace` as shown in Fig. 6.1.

```
$ xmgrace Si2.adaptive.agr.
```

5. We now try again with a better sampling of the DOS, by setting `DOS_SPACING : 0.001` and also analyse the band gap, by setting `COMPUTE_BAND_GAP : true`. You can remove all of the OPTADOS output files by using `./tools/optados_clean` in your working directory. If you have a parallel version of OPTADOS compiled, now might be the time to try it out, if not, the serial version will be fine, but just take a bit longer. You can set `IPRINT : 2` to see a progress report in `Si2.odo`. In parallel:

```
$ mpirun -np <nprocs> optados.SYSTEM.BUILD.COMMS_ARCH.x86_64 Si2
```

but your MPI implementation may be different.

6. In `Si2.odo` we now have a new section analysing the band gap in various ways.

```

+----- Bandgap Analysis -----+
|          Number of kpoints at          VBM          CBM          |
|          Spin :   1   :           1           1           |
|          Spin :   2   :           1           1           |
+-----+

```

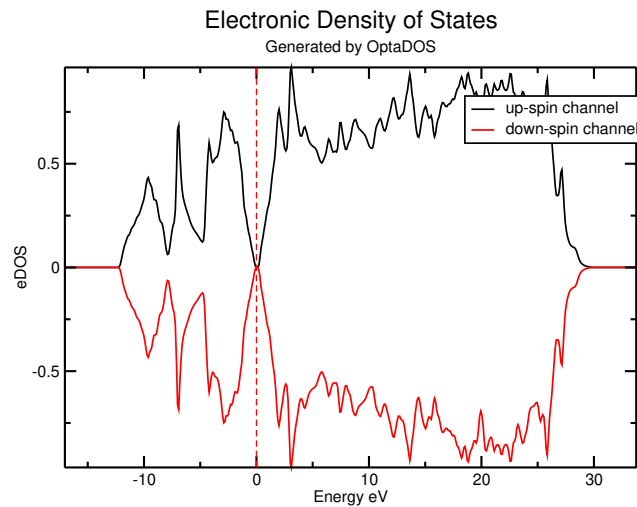


Figure 6.1: Density of States of Silicon generated by adaptive broadening and a very coarse energy sampling of 0.1 eV.

Thermal Bandgap :	0.6676272107	eV	<- TBg
Between VBM kpoint :	0.05000	0.05000	0.05000
and CBM kpoint:	-0.45000	-0.05000	-0.45000
==> Indirect Gap			

Optical Bandgap			
Spin : 1 :	2.5542517447	eV	<- OBg
Spin : 2 :	2.5542463024	eV	<- OBg
Number of kpoints with this gap			
Spin : 1 :	1		
Spin : 2 :	1		

Average Bandgap			
Spin : 1 :	3.8121372691	eV	<- ABg
Spin : 2 :	3.8121342659	eV	<- ABg
Weighted Average :	3.8121357675	eV	<- wAB

OPTADOS is very careful in its band gap analysis. It uses the bare eigenvalues (unbroadened) and works out the nature and size of the thermal gap, optical gap and the average gap over all of the Brillouin zone. In cases of multi-valleyed semiconductors OPTADOS will report the number of conduction band minima or valence band maxima with identical energies, but will not report the nature of the gap.

Increasing the number of integration points has improved the band energy of the adaptive smearing:

Band energy (Adaptive broadening) :	1.3623	eV	<- BEA
-------------------------------------	--------	----	--------

7. Now set TASK : `compare_dos` and re-run OPTADOS. OPTADOS will calculate DOS using all the broadening methods, this is good practice to see whether the broadening widths are appropriate before more advanced tasks are carried out, such as Joint-DOS, core and optical calculations.

Plotting the linear broadened DOS over the adaptive we see that the default adaptive broadening is appropriate,

```
xmgrace Si2.adaptive.agr -nxy Si2.linear.dat
```

Linear broadening, although a massive improvement over fixed broadening, sometimes appears noisy if used with very low numbers of k-points. Linear and adaptive DOS should be compared and ADAPTIVE_SMEARING may be tuned by eye until the adaptive DOS contains the features of the linear DOS, but with less noise. Adding a random shift to a k-point mesh can greatly increase the quality of the DOS, especially if the k-point set contained the Γ -point. It generally pays off in computational time to have a coarse mesh at low symmetry points, than a fine mesh centred on high symmetry points.

Both fixed and adaptive broadening can fail to plot the sharpest features, such as semi-core states, if the bin widths are too broad. These sharp features may be forced to be present using the narrowest Gaussian still reproducible by the chosen bin widths by setting FINITE_BIN_CORRECTION : `true`.

Linear broadening may also be improved by using HYBRID_LINEAR : `true`. Van Hove singularities and other sharp features are now described at the adaptive broadening level if the bands are flatter than HYBRID_LINEAR_GRAD_TOL. Hybrid linear may also take advantage of the finite bin correction if required.

8. Compare the fixed and adaptive DOS and see the advantage of adaptive broadening over standard Gaussian smearing.

6.2 Projected Density of States

We assume the reader is familiar with the previous section on Density of States calculations and is now familiar with choosing broadening widths, and running OPTADOS.

- *Outline* : This is a simple example of using OPTADOS for calculating electronic density of states of 2 atoms of crystalline silicon projected onto LCAO basis states.
- *Input Files*:
 - `examples/Si2_PDOS/Si2_dos.cell` - The CASTEP cell file containing information about the simulation cell.
 - `examples/Si2_PDOS/Si2_dos.param` - The CASTEP param file containing information about the parameters for the SCF and spectral calculations.
 - `examples/Si2_PDOS/Si2_dos.odi` - The OPTADOS input file, containing the parameters necessary to run OPTADOS.

1. Choose a broadening scheme for the projected-DOS calculation and test using TASK : `compare_dos` as explained in the previous example. Checking that the DOS_SPACING is sufficiently fine for the band energies to match.
2. Once the DOS looks suitable, switch to TASK : `pdos`. We choose to decompose the DOS into angular momentum channels (PDOS : `angular`) and as in the previous example we choose to recalculate the Fermi level using the calculated DOS, rather than use the Fermi level suggested by CASTEP.
3. Execute OPTADOS.
4. The output can be found in `Si2.pdos.dat`.

```
#####
#
#               O p t a D O S   o u t p u t   f i l e
#
#   Generated on 13 Feb 2012 at 10:15:10
#####
#+-----+
#|               Partial Density of States -- Projectors               |
#+-----+
#| Projector:    1 contains:                                           |
#|      Atom      AngM Channel                                         |
#|      Si   1      s                                                  |
#|      Si   2      s                                                  |
#+-----+
#| Projector:    2 contains:                                           |
#|      Atom      AngM Channel                                         |
#|      Si   1      p                                                  |
#|      Si   2      p                                                  |
#+-----+
#| Projector:    3 contains:                                           |
#|      Atom      AngM Channel                                         |
#|      Si   1      d                                                  |
#|      Si   2      d                                                  |
#+-----+
#| Projector:    4 contains:                                           |
#|      Atom      AngM Channel                                         |
#|      Si   1      f                                                  |
#|      Si   2      f                                                  |
#+-----+
```

The header shows that there are four projectors described below. The first containing the s-channels of both silicon atoms, the second the p-channels *etc.*

5. The output is easily plotted using `xmgrace`:
`xmgrace -nxy Si2.pdos.dat`

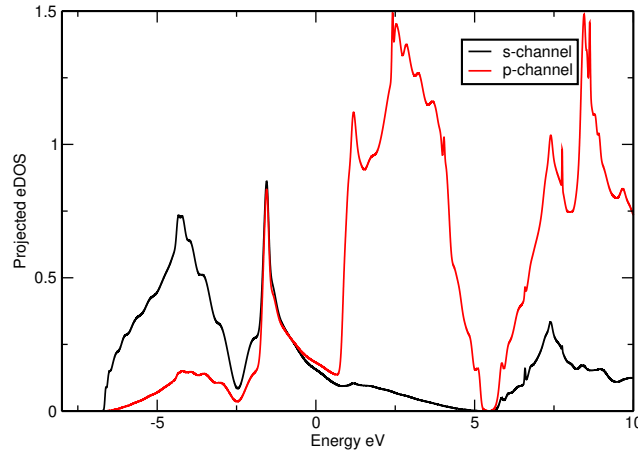


Figure 6.2: Density of States of Silicon generated by adaptive broadening projected onto LCAO momentum states.

6. Setting `DOS_SPACING : 0.001` gives a high quality plot, as shown in Fig 6.2

7. Other things to try are:

- `PDOS : Si1;Si2(s)` – Output the PDOS on Si atom 1 and the PDOS on the s-channel of Si atom 2. (Resulting in two projectors)
- `PDOS : sum:Si1-2(s)` – Output the sum of the s-channels on the two Si atoms. (Resulting in one projector)
- `PDOS : Si1(p)` – Output the p-channel on Si atom 1. (Resulting in one projector)

6.3 Projected Bandstructure

- *Outline* : This is a simple example of using OPTADOS to calculate the electronic bandstructure of 2 atoms of crystalline silicon projected onto LCAO basis states. The process is almost identical to the PDOS example above, but requires performs the projections on a continuous k-point path rather than a grid.

- *Input Files*:

- `examples/Si2_PDIS/Si2_pdis.cell` - The CASTEP cell file containing information about the simulation cell.
- `examples/Si2_PDIS/Si2_pdis.param` - The CASTEP param file containing information about the parameters for the SCF and spectral calculations.
- `examples/Si2_PDOS/Si2_pdis.odi` - The OPTADOS input file, containing the parameters necessary to run OPTADOS.

1. Run CASTEP in the same way as previous examples, ensuring that `PDOS_CALCULATE_WEIGHTS: TRUE` is set in the CASTEP parameter file.

2. Run OPTADOS with the desired projections, following the same format as PDOS in `Si2_pdis.odi`.
3. The output can be found in `Si2_pdis.dat`, which contains the weight of each projector at each eigenvalue, for each k-point. Plotting this data in an attractive way is tricky; you might consider trying the Python package `matador` (www.matador.science).

6.4 JDOS

See `examples/Si2_JDOS/`. This is a simple example of using OPTADOS for calculating joint electronic density of states. We choose to recalculate the Fermi level using the calculated DOS, rather than use the Fermi level suggested by CASTEP and so `EFERMI`: OPTADOS is included in the `Si2.odi` file.

1. Execute OPTADOS using the example files. The JDOS is outputted to `Si2.jadaptive.dat`. A file suitable for plotting using `xmgrace` is written to `Si2.jadaptive.gr`.
2. Check the effect of changing the sampling by increasing and decreasing the value of `JDOS_SPACING` in the `Si2.odi` file.
3. If `TASK : compare_jdos` is used instead, OPTADOS will calculate the JDOS using all the broadening methods. This is good practice to see whether the broadening widths are appropriate before more advanced tasks are carried out.

6.5 OPTICS

Two sets of example files are provided for calculations of optical properties. For each example, the CASTEP files containing all the cell and simulation parameters are included, along with an OPTADOS input file. We assume that the reader is familiar with the previous sections on DOS and JDOS.

- `examples/Si2_OPTICS/` This is a simple example of using OPTADOS to calculate the optical properties of crystalline silicon, which is an insulator.
- `examples/Al_OPTICS/` This is a simple example of using OPTADOS to calculate the optical properties of a metal, aluminium.

6.5.1 Silicon

1. Choose a broadening scheme for the JDOS as explained in the JDOS example. Once the JDOS looks suitable, switch the `task` from `JDOS` to `OPTICS` and execute OPTADOS to calculate the optical properties. Several `*.dat` files are produced:
 - `Si2_OPTICS_absorption.dat` : This file contains the absorption coefficient (second column) as function of energy (first column).

- `Si2_OPTICS_conductivity.dat` : This file contains the conductivity outputted in SI units (Siemens per metre). The columns are the energy, real part and imaginary part of the conductivity respectively.
- `Si2_OPTICS_epsilon.dat` : This file contains the dielectric function. The columns are the energy and real and imaginary parts of the dielectric function respectively. The file header also includes the result of the sum rule $\int_0^{\omega'} \text{Im}\epsilon(\omega) d\omega = N_{eff}(\omega')$. N_{eff} is the effective number of electrons contributing to the absorption process, and is a function of energy.
- `Si2_OPTICS_loss_fn.dat` : This file contains the loss function (second column) as a function of energy (first column). The header of the file shows the results of the two sum rules associated with the loss function $\int_0^{\omega'} \text{Im}\left(\frac{-1}{\epsilon(\omega)}\right) \omega d\omega = N_{eff}(\omega')$ and $\int_0^{\omega'} \text{Im}\left(\frac{-1}{\epsilon(\omega)}\right) \frac{1}{\omega} d\omega = \frac{\pi}{2}$.
- `Si2_OPTICS_reflection.dat` : This file contains the reflection coefficient (second column) as a function of energy (first column).
- `Si2_OPTICS_refractive_index.dat` : This file contains the refractive index. The columns are the energy and real and imaginary parts of the refractive index respectively.

Corresponding `*.agr` files are also generated which can be plotted easily using `xmgrace`.

2. Change parameters `JDOS_SPACING` and `JDOS_MAX` and check the effect on the optical properties. Note: all of the other optical properties are derived from the dielectric function.
3. The OPTADOS input file has been set up to calculate the optical properties in the polycrystalline geometry (`optics_geom = polycrystalline`). It is possible to calculate either polarised or unpolarised geometries, or to calculate the full dielectric tensor. To calculate the full dielectric tensor set `optics_geom = tensor`. This time only the file `Si2_OPTICS_epsilon.dat` is generated. The format of this file is the same as before (the columns are the energy and the real and imaginary parts of the dielectric function respectively), but this time the six different components of the tensor are listed sequentially in the order ϵ_{xx} , ϵ_{yy} , ϵ_{zz} , ϵ_{xy} , ϵ_{xz} and ϵ_{yz} .
4. Additional broadening can be included in the calculation of the loss function. This is done by including the keyword `optics_lossfn_broadening` in the OPTADOS input file. If you include this keyword and re-run OPTADOS, you will find that the file `Si2_OPTICS_loss_fn.dat` now has three columns. These are the energy, unbroadened spectrum and broadened spectrum respectively.

6.5.2 Aluminium

1. As for the silicon example, a broadening scheme for the JDOS should first be determined.
2. Aluminium is a metal so we need to include both the interband and intraband contributions to the dielectric function. To include the intraband contribution `optics_intraband`

`= true` must be included in the OPTADOS input file. When you run OPTADOS the same files are generated as when only the interband term is included.

The `Al_OPTICS_epsilon.dat` file has the same format as before, but it now contains sequentially the interband contribution, the intraband contribution and the total dielectric function. (The file `Al_OPTICS_epsilon.agr` only contains the interband term.) In the same way, `Al_OPTICS_loss_fn.dat` contains the interband contribution, intraband contribution and total loss function. All other optical properties are calculated from the total dielectric function and the format of the output files remains the same.

3. In the case where the dielectric tensor is calculated and the intraband term is included, only the `Al_OPTICS_epsilon.dat` file is generated. As before it contains each component, but this time it lists sequentially the interband contribution, intraband contribution and total dielectric function for each component.
4. This time, if additional broadening for the loss function is included by using the key word `optics_lossfn_broadening`, `Al_OPTICS_loss_fn.dat` will contain four sequential data sets. These are the interband contribution, the intraband contribution, the total loss function without the additional broadening and the broadened total loss function.

6.6 CORE

See `examples/Si2_CORE/`. This is a simple example of using OPTADOS for calculating core level absorption spectra for crystalline silicon. We assume that the reader is familiar with the previous section on calculating DOS.

1. We begin by running a CASTEP calculation using the files provided in `examples/Si2_CORE`. Note that we do not specify pseudopotentials in the `Si2_CORE.cell` file hence require CASTEP to generate on-the-fly pseudopotentials. This is important as most pseudopotential formats do not contain enough information for the PAW reconstruction needed for a CORE calculation. For any atom a CORE spectra is to be calculated an On-the-fly pseudopotential must be used.

Execute OPTADOS using the OPTADOS input file provided and the file `Si2_CORE_core_edge.dat` will be created. The file contains two columns, the first is the energy and the second is the spectrum. This file contains the following edges:

```
# Si 1    K1
# Si 1    L1
# Si 1    L2,3
# Si 2    K1
# Si 2    L1
# Si 2    L2,3
```

i.e. all edges from all atoms are produced.

2. To include a core-hole in the calculation, first one atom is chosen to have the excitation. To begin, we will keep two atoms in the unit cell and distinguish one atom by changing the `%BLOCK POSITIONS_FRAC`

```
%BLOCK POSITIONS_FRAC
      Si:exi    0.0000000000    0.0000000000    0.0000000000
      Si        0.2500000000    0.2500000000    0.2500000000
%ENDBLOCK POSITIONS_FRAC
```

in the `Si2_CORE.cell` file. The atom `Si:exi` is the one to have a core-hole. To create a core-hole we remove a 1s electrons from the electronic configuration used in the generation of the pseudopotential. We have already generated an on-the-fly pseudopotential without a core-hole in the previous section. Information about the pseudopotentials is included at the top of the `Si2_CORE.caste` file.

```
=====
| Pseudopotential Report - Date of generation 16-05-2012 |
|-----|
| Element: Si Ionic charge: 4.00 Level of theory: LDA |
| |
|           Reference Electronic Structure |
|           Orbital      Occupation      Energy |
|           3s           2.000          -0.400 |
|           3p           2.000          -0.153 |
| |
|           Pseudopotential Definition |
|           Beta      l      e      Rc      scheme      norm |
|           1         0     -0.400  1.797      qc         0 |
|           2         0      0.250  1.797      qc         0 |
|           3         1     -0.153  1.797      qc         0 |
|           4         1      0.250  1.797      qc         0 |
|           loc        2      0.000  1.797      pn         0 |
| |
| Augmentation charge Rinner = 1.298 |
| Partial core correction Rc = 1.298 |
|-----|
| "2|1.8|1.8|1.3|2|3|4|30:31:32LGG(qc=4)" |
|-----|
|           Author: Chris J. Pickard, Cambridge University |
|=====
```

The line

```
2|1.8|1.8|1.3|2|3|4|30:31:32LGG(qc=4)
```

specifies the parameters used to create the pseudopotential. We use this as the starting point and then remove one of the core 1s electrons to create a core-hole pseudopotential. This is done by including `{1s1.00}` in the pseudopotential string as shown:

```
2|1.8|1.8|1.3|2|3|4|30:31:32LGG{1s1.00}(qc=4)
```

If, instead of removing a 1s electron, we wanted to remove a 2s electron from the core, we would have included {2s1.00} instead of {1s1.00} in the pseudopotential string.

We are only interested in the spectra from the atom with the core-hole and so copy the pseudopotential file generated by the previous calculation (Si_OFT.usp) to Si_LDA.usp. Then include

```
%BLOCK SPECIES_POT
  Si:exi    2|1.8|1.8|1.3|2|3|4|30:31:32LGG{1s1.00}(qc=4)
  Si        Si_LDA.usp
%ENDBLOCK SPECIES_POT
```

in the CASTEP Si2_CORE.cell file.

To maintain the neutrality of the cell, we include

```
CHARGE : +1
```

in the Si2_CORE.param file. Run the calculation. This time the Si2_CORE_core_edge.dat file will contain only the edges from the core-hole atom. Compare the K-edge from the core-hole calculation with the previous non-core-hole calculation.

3. The periodic images of the core-hole will interact with one another. As this is unphysical, we need to increase the distance between the core-holes. This is done by creating a supercell. To start with we use a face-centred unit cell rather than the primitive unit cell. This is done by changing the lattice parameters and fractional co-ordinates to:

```
%BLOCK LATTICE_CART
  5.46  0.00  0.00
  0.00  5.46  0.00
  0.00  0.00  5.46
%ENDBLOCK LATTICE_CART

%BLOCK POSITIONS_FRAC
  Si:exi    0.0000000000    0.0000000000    0.0000000000
  Si        0.5000000000    0.5000000000    0.0000000000
  Si        0.5000000000    0.0000000000    0.5000000000
  Si        0.0000000000    0.5000000000    0.5000000000
  Si        0.2500000000    0.2500000000    0.2500000000
  Si        0.7500000000    0.2500000000    0.7500000000
  Si        0.2500000000    0.7500000000    0.7500000000
  Si        0.7500000000    0.7500000000    0.2500000000
%ENDBLOCK POSITIONS_FRAC
```

Run OPTADOS and compare the spectrum from the face-centred unit cell with that from the primitive unit cell. Continue constructing larger unit cells until the core-hole spectrum stops changing with increasing separation between the periodic images.

4. Other things to try include
 - Changing the geometry from polycrystalline to polarised
 - Including life-time and instrumentation broadening

6.7 CORE with Mizoguchi Chemical Shift

In the previous section we discussed including a core-hole in the `seedname.cell` file. In order to account for the energetic contribution of the core-hole to the absorption or emission edge users may find it useful to include the chemical shift of the energy term as described by Mizoguchi *et al.* [10]. The Mizoguchi shift, rescales the absorption or emission energy by calculating the energy difference between the ground state energy with and without the core-hole included in the input `seedname.cell` file. See `examples/Mizoguchi` for input files described here.

1. We begin by running a CASTEP calculation using the files provided in `examples/Mizoguchi`. We have included a 1s core hole on the Ge atom in the cell in order to obtain a K-edge spectrum. This is specified by adding a `Ge:exc` label in the `% BLOCK POSITIONS_FRAC` and by including a core hole in the pseudopotential string as shown here:

```
%BLOCK positions_frac
Na 0.620573953121250      0.758852093757499      0.836258492531341
Na 0.379426046878750      0.241147906242501      0.163741507468659
Na 0.220453843197638      0.559092313604724      0.600726924771292
Na 0.779546156802362      0.440907686395276      0.399273075228708
Ge:exc 0.000000000000000  0.000000000000000  0.000000000000000
%ENDBLOCK positions_frac

%BLOCK species_pot
Ge:exc 3|1.5|13|15|17|30U:31U:32:40:41(qc=6){1s1}
Na 2|1.3|14|16|18|20U:30:21(qc=7)
%ENDBLOCK species_pot
```

Once the CASTEP calculation is completed, run OPTADOS and look at the output. You will see that the energy range of the spectrum is from 0 eV to 30 eV, as we have not yet applied the Mizoguchi chemical shift term

2. Next, we want to add in the shift which accounts for the relative energy of our cell with a core-hole to that without a core hole. It is best to rename your `NaGe.cell` and `NaGe.param` files so that they are not overwritten in the next CASTEP calculation. We will now get the ground state energy of the cell without a core hole included. To do this, we will remove the specification for our pseudopotential which included a core hole so that your cell file (renamed to something like `NaGe-singlepoint.cell`) looks like this:

```
%BLOCK positions_frac
Na 0.620573953121250      0.758852093757499      0.836258492531341
Na 0.379426046878750      0.241147906242501      0.163741507468659
Na 0.220453843197638      0.559092313604724      0.600726924771292
Na 0.779546156802362      0.440907686395276      0.399273075228708
Ge 0.000000000000000      0.000000000000000      0.000000000000000
%ENDBLOCK positions_frac

%BLOCK species_pot
```

```

Ge 3|1.5|13|15|17|30U:31U:32:40:41(qc=6)
Na 2|1.3|14|16|18|20U:30:21(qc=7)
%ENDBLOCK species_pot

```

To run a singlepoint calculation, we want to remove the CHARGE : +1 that we added in the .param file and change the task to task : singlepoint so your NaGe-singlepoint.param file should look like:

```

task : singlepoint
cut_off_energy : 250 eV
xc_functional : PBE
! charge : +1
write_checkpoint : None

```

Here we have commented out the line for charge inclusion, and changed the task keyword. Run the CASTEP calculation and compare the ground state total energies of your two different files. You should see that the groundstate energy of -11848.48889250 eV is less than the core hole groundstate energy.

3. Now we want to calculate the Mizoguchi chemical shift associated with this difference in energies. To do this, we will use the code located at tools/miz_chemical_shift. This code is written in PYTHON and you can see the different keywords it takes using the --help flag. To calculate the shift for our system we will use the command:

```

../../tools/miz_chemical_shift -i NaGe -e Ge -s NaGe-singlepoint

```

Or alternatively:

```

../../tools/miz_chemical_shift -i NaGe -e Ge -t -11848.48889250

```

Where we have specified the location of the miz_chemical_shift script, the name of the file we performed the core-hole calculation on -i NaGe, the element we performed our calculation on -e Ge and either the file with the singlepoint calculation -s NaGe-singlepoint or the value of the singlepoint energy without a core hole -t -11848.48889250. **NOTE:** The miz_chemical_shift script is written assuming that you are providing the CASTEP NB est. 0K energy. If it cannot find the NB est. 0K energy then it will look for the Final energy. Therefore it is important if for example your calculation includes a dispersion correction or Hubbard U parameter which affects the final ground state energy to provide the energy before that correction is applied. This way the -t flag will reflect the same energy as the core-hole final energy. If you use the -s flag and get the final ground state energy from a file, miz_chemical_shift will explicitly state which energy it is using in the output mizoguchi.out file. Run this command and take a look at your NaGe.odi file. You will see a line has been added:

```

core_chemical_shift : 11142.275218

```

Now when we run OPTADOS again, the energy scale on the x-axis will be shifted by the amount specified in your input file for OPTADOS. This is especially useful when comparing your results to experimental data, as it will allow you to apply a shift to your output from optados into the experimentally relevant energy range. For reference, the known experimental absorption edge for Ge is 11.1031 eV, very close to the shift calculated using the Mizoguchi method. In addition, this is a useful tool when calculating this shift across different core-hole sites in your cell, when your cell has more than one symmetry-inequivalent atom, such that the relative absorption energies of each atom are accounted for.

6.8 Photoemission

This is an example of using photoemission module for calculating the photoemission properties of Cu(100) surface using a 16 atomic layers slab.

- Input Files

- `examples/Cu_photo/Cu.cell` - The CASTEP cell file containing information about the simulation cell.
- `examples/Cu_photo/Cu.param` - The CASTEP parameter file containing information about the parameters for the SCF and spectral calculations.
- `examples/Cu_photo/Cu.odi` - The OPTADOS input file, containing the parameters necessary to run OPTADOS.

1. Perform a CASTEP calculation using the `Cu.cell` and `Cu.param` input files. Slab calculations are time-consuming calculations, a high performing computer should be used.

```
$ castep Cu
```

More help can be found in the tutorials on the CASTEP website www.castep.org.

2. Perform an OPTADOS calculation.

```
$ optados.x Cu
```

This generates several files:

- `Cu.odo` – OPTADOS general output file.
- `Cu_bindenergy_curve.dat` – The Gaussian broadened angle-resolved binding energy spectra raw output data.

3. Open the `Cu.odo` file in a text editor (*e.g.* `vi` or `emacs`). The relevant OPTADOS parameters for the optical properties and photoemission are printed. The input file provided creates the following output of photoemission parameters:

```
+----- PHOTOEMISSION PARAMETERS -----+
| Photoemission Model                      : 3-Step Model |
```

Photoemission Final State	:	Bloch State	
*** Including transmission probability across surface ***			
Photon Energy (eV)	:	5.0000	
Work Function (eV)	:	4.5430	
Slab Max Z-Coord. (Ang)	:	43.2730	
Slab Min Z-Coord. (Ang)	:	13.4180	
IMFP Constant (Ang)	:	5.3000	
Bulk cutoff dist. (int. multiple of IMFP)	:	10.0	
Electric Field Strength (V/m)	:	0.0000	
Smearing Temperature (K)	:	300.00	
Transverse Momentum Scheme	:	crystal	
Theta - min - (deg)	:	0.00	
Theta - max - (deg)	:	90.00	
Phi - min - (deg)	:	0.00	
Phi - max - (deg)	:	90.00	
Binding Energy Broad. Width (eV)	:	0.0257	

-----+

-----+ Electronic Data -----+

Number of Bands	:	190	
Grid size	:	27 x 27 x 1	
Number of K-points	:	105	
Spin-Polarised Calculation	:	False	
Number of electrons	:	176.00	

-----+

By default OPTADOS calculates the Fermi energy from the supplied bands file and the calculated DOS, but can also be supplied by the user. An exemplary output for the calculated Fermi energy this is as follows:

-----+ Fermi Energy Analysis -----+

From Adaptive broadening	
Spin Component : 1 occupation between 175.99388 and 176.00354	<- Occ
Fermi energy (Adaptive broadening) :	0.1564 eV <- EfA

-----+

Fermi energy from DOS :	0.1564 eV <- EfD
-------------------------	------------------

-----+

Since we had `efermi : optados`, OPTADOS sets the internal value of the Fermi level to the one it has derived from the DOS. In a slab model, the top and bottom surfaces are equivalent by symmetry and therefore, only the top half is used for the photoemission calculation.

-----+ Atomic Order -----+

Atom	Atom Order	Box/Layer	Atom Z-Coordinate (Ang)	
------	------------	-----------	-------------------------	--


```

+----- Bulk Approximation Slab Info -----+
|   Number Bulk layers =    29               |
|   Vol. per layer      =    11.3363          |
|   Total Volume        =   328.7518          |
+---- P_esc values for an electron with E = E_fermi and E_transverse = 0 ----+
|   Layer #   1   I_light = 0.517860E+00 P_esc = 0.688302E-01 |
|   Layer #   2   I_light = 0.501957E+00 P_esc = 0.491400E-01 |
|   Layer #   3   I_light = 0.479013E+00 P_esc = 0.350825E-01 |
|                                     ..... |
|   Layer #  27   I_light = 0.144792E-02 P_esc = 0.107853E-04 |
|   Layer #  28   I_light = 0.935618E-03 P_esc = 0.769998E-05 |
|   Layer #  29   I_light = 0.595220E-03 P_esc = 0.549725E-05 |
+-----+

```

Finally the layer QE, total QE and MTE are calculated and printed:

```

+-----+
| Work Function          4.5430 eV           Photon Energy    5.0000 eV |
| Effective Work Function 4.5430 eV           Electric Field   0.0000 V/m |
| Final State : Bloch State                                     |
+-----+
| Atom | Atom Order | Layer | Quantum Efficiency |
| Cu    1      1      0.1220E-006 |
| Cu    2      2      0.3016E-017 |
| Cu    3      3      0.5586E-008 |
| Cu    4      4      0.9952E-008 |
| Cu    5      5      0.1400E-007 |
| Cu    6      6      0.9265E-008 |
| Cu    7      7      0.4518E-008 |
| Cu    8      8      0.1398E-008 |
| Bulk                                0.4176E-007 |
| Total Quantum Efficiency (electrons/photon): 0.2085E-006 |
| Weighted Mean Transverse Energy (eV):       0.3414E-001 |
+-----+

```

With debug printing `iprint = 3` many more digits are printed.

4. With the option `photo_output : bindenergy_curve` a QE vs binding energy curve, also known as energy distribution curve (EDC) for ARPES experiments will be printed. To visualise this file, use: `xmgrace -nxy Cu_bindenergy_curve.dat`.

Chapter 7

ONETEP Examples

7.1 (L)PDOS

ONETEP outputs an OPTADOS compatible `seedname_prefix.bands` file, and from version 4.2 onwards, a `seedname_prefix.pdos_bin`, `seedname_prefix.dos_bin` and a `seedname-out.cell` file. Where `prefix` may be one of `val`, `cond` or `joint`. A normal ONETEP run will only output the valence files (`val`), if conduction states are required, a ONETEP conduction calculation should be run, in which case the `cond` and `joint` files will also be written to disk. For a full DOS (conduction and valence), the `joint` files should be used with OPTADOS.

When running ONETEP, in order to output OPTADOS compatible files, `DO_PROPERTIES` must be set in the input file or a properties calculation must be run in continuation of a prior, converged calculation.

`PDOS_OPTADOS_OUTPUT` should be set to `TRUE`, this is the default value.

`PDOS_MAX_L=integer`, where `integer` specifies the maximum angular momentum spherical wave to project onto, i.e. `s=0`, `p=1`, `d=2`, etc.

`DOS_SMEAR` should be set to a value greater than 0.0. When `DOS_SMEAR>0.0`, then the `seedname_prefix.bands` file will be written to disk, if `PDOS_MAX_L>=0`, then a PDOS calculation will be performed and the associated files (mentioned above) will also be written to disk.

When outputting the PDOS weights in `seedname_prefix.pdos_bin`, ONETEP will also output a text file called `pdos_weights.dat` which is not associated with OPTADOS but meant for manual inspection of the PDOS weights.

ONETEP does not produce an OPTADOS input file, this should be the only additional file (besides an OPTADOS binary) required to perform a ONETEP +OPTADOS (L)PDOS calculation.

All of the DOS functionality in ONETEP is also PAW compatible, though it is not mandatory.

7.2 EELS/ELNES

ONETEP can also produce OPTADOS compatible `seedname.elnes_bin` files by asking for an EELS[22] (conduction calculation) properties post-processing calculation. This is done by specifying `COND_CALC_EELS=TRUE` in the ONETEP input file.

To perform an EELS calculation, the PAW method must be used (`PAW=TRUE` specified in the input file and PAW datafiles for each species). For those species which the user wishes to probe core to conduction transitions, core wavefunction data must also be supplied.

Finally, for EELS calculations, a conduction calculation must be performed, so `COND_ENERGY_RANGE`, which specifies the energy window above the HOMO in which conduction states will be optimised and `COND_ENERGY_GAP`, which specifies the energy gap between the highest optimised state and the lowest un-optimised state must be set in the ONETEP input file.

More can be found on EELS/ELNES here: ¹

¹http://www2.tcm.phy.cam.ac.uk/onetep/pmwiki/uploads/Main/Documentation/eels_in_onetep.pdf

Chapter 8

Frequently Asked Questions

8.1 OPTADOS crashes complaining that it can't read the seed.bands or seed.cst_ome file.

Which version of CASTEP are you using? See Sect. 5.2.5.

8.2 The examples do not work and I'm using CASTEP version 6.0

CASTEP version 6.0 was released during the transition of the spectral module and features. The easiest way to solve this is to obtain a copy of CASTEP > 6.0, where OPTADOS will work "out of the box". However, to make the examples work with CASTEP 6.0 the OPTADOS examples require a few tweaks outlined below.

- Replace SPECTRAL_KPOINTS_MP_GRID in the cell file with BS_KPOINTS_MP_GRID and OPTICS_KPOINTS_MP_GRID
- Remove SPECTRAL_TASK : DOS from the param file.
- Add DEVEL_FLAG:old_filename to the odi file.
- Add WRITE_CELL_STRUCTURE : true to your CASTEP cell file to generate the symmetries of the cell if you are doing pdos, core or optics calculations.
- Read Section 8.3 if you are planning to do a partial density of states calculation.

8.3 I'm getting odd results for my PDOS calculation when using CASTEP version 6.0

The spectral module in CASTEP 6.0 generates the pdos weights commensurate with the optics grid you have chosen. However this gets overwritten by the main routines. Hence to get the right PDOS weights you must set PDOS_CALCULATE_WEIGHTS : FALSE in the param file.

8.4 OPTADOS is reporting the wrong number of kpoints

This is a rare bug that can happen if you have picked the magic k-point offset for your grid, that is a $3/(4n)$ offset where n is the number of MP grid points in that direction. It is a very difficult edge-case to catch. Until a permanent workaround is found you can tell OPTADOS the right number using the KPOINT_MP_GRID nx ny nz in the odi file.

8.5 I'd like OPTADOS to do X as well

Contact the developers we're always interested in discussing new functionality.

8.6 I'd like to help, what can I do?

Contact the developers, there's always more functionality that we'd like to add to the code.

8.7 I think I've found a bug: what should I do?

- Check and re-check that it is a bug.
- Check the output of the electronic structure code.
- Check that you're using the latest version of OPTADOS.
- Email the developers the input and output files with `iprint : 3` and as much information about the problem as possible.

Appendix A

Interface with other codes

Currently OPTADOS is interfaced with CASTEP and ONETEP. This appendix consists of Fortran95 code which defines the input files for OPTADOS so as to aid its interfacing with other electronic structure codes. Presented below are the type declaration statements and write statements that may be used in other electronic structure codes or converters to generate the .bands, .ome_bin, .pos_bin and .elnes_bin files used as inputs to OPTADOS. These code snippets are also available in the `tools/` directory of the OPTADOS distribution.

A.1 .bands file

The .bands file is necessary for any calculation using OPTADOS it contains the energy eigenvalues at each k-point and data about the electronic properties of the system. The bands file is a formatted file.

```
integer,parameter:: dp=selected_real_kind(15,300) ! Define double precision
integer:: num_kpoints          ! Number of kpoints
integer:: num_spins            ! Number of spins
integer:: max_eigenv           ! Number of bands included in matrix elements
integer:: num_electrons(num_spins) ! Number of electrons of each spin
integer:: num_eigenvalues(num_spins) ! Number of eigenvalues of each spin
real(dp):: efermi(num_spins)    ! Fermi level calculated from electronic
                                ! structure code for each spin
real(dp):: cell_lattice(1:3,1:3) ! Real-space lattice vectors
real(dp):: kpoint_positions(num_kpoints,1:3) ! K-point position vectors in
                                                ! fractions of Brillouin Zone
real(dp):: kpoint_weight(num_kpoints) ! K-point weight
real(dp):: band_energy(max_eigenv, num_spins, num_kpoints) ! Energy eigenvalue
                                                            ! list

write (band_unit,'(a,i4)') 'Number of k-points ', num_kpoints
write (band_unit,'(a,i1)') 'Number of spin components ', num_spins
if(num_spins>1) then
```

```

    write (band_unit,'(a,2g10.4)'), 'Number of electrons ', num_electrons(:)
else
    write (band_unit,'(a,g10.4)'), 'Number of electrons ', num_electrons(:)
endif
if(num_spins>1) then
    write (band_unit,'(a,2i4)'), 'Number of eigenvalues ', num_eigenvalues(:)
else
    write (band_unit,'(a,i4)'), 'Number of eigenvalues ', num_eigenvalues(:)
endif
if(num_spins>1) then
    write (band_unit,'(a,2f12.6)') 'Fermi energy (in atomic units) ', efermi(:)
else
    write (band_unit,'(a,f12.6)') 'Fermi energies (in atomic units) ', efermi(:)
endif
write(band_unit,'(a)') 'Unit cell vectors'
write(band_unit,'(3f12.6)') cell_lattice(1,:)
write(band_unit,'(3f12.6)') cell_lattice(2,:)
write(band_unit,'(3f12.6)') cell_lattice(3,:)

do ik=1,num_kpoints
    write (band_unit,'(a,i4,4f12.8)') 'K-point ', ik, &
        & kpoint_positions(num_kpoints,1:3), kpoint_weight(num_kpoints)
    do is=1,num_spins
        write(band_unit,'(a,i1)') 'Spin component ',is
        do ib=1,num_eigenvalues(is)
            write (band_unit,*) band_energy(ib,is,ik)
        end do
    end do
end do
end do

```

A.2 .ome_bin file

The .ome_bin file is required for adaptive and linear broadening, and for optics calculations. Note that a .ome file is just the diagonal elements of the .ome. The .ome_bin file is an unformatted file. Note that, in line with many other electronic structure codes, OPTADOS reads in the momentum matrix elements, $\langle \psi_{\mathbf{k}}^c | \mathbf{p} | \psi_{\mathbf{k}}^v \rangle$, and converts them to OMEs internally.

```
integer,parameter:: dp=selected_real_kind(15,300) ! Define double precision
real(dp):: file_version=1.0_dp ! File version
character(len=80):: file_header ! File header comment
integer:: num_kpoints ! Number of k-points
integer:: num_spins ! Number of spins
integer:: max_eigenv ! Number of bands included in matrix elements
integer:: num_eigenvalues(1:num_spins) ! Number of eigenvalues per spin channel
complex(dp):: optical_mat(max_eigenv,max_eigenv,1:3,1:num_kpoints,num_spins)! OMEs

write(ome_unit) file_version
write(ome_unit) file_header

do ik=1,num_kpoints
do is=1,num_spins
write(ome_unit) (((optical_mat(ib,jb,i,ik,is),ib=1,num_eigenvalues(is)), &
&jb=1,num_eigenvalues(is)),i=1,3)
end do
end do
```

A.3 .pdos_bin file

The .pdos_bin file is required for PDOS calculations. The .pdos_bin file is an unformatted file.

```
integer,parameter:: dp=selected_real_kind(15,300) ! Define double precision
integer:: num_kpoints      ! Number of k-points
integer:: num_spins        ! Number of spins
integer:: num_popn_orb     ! Number of LCAO projectors
integer:: max_eigenv       ! Number of bands included in matrix elements
real(dp):: file_version=1.0_dp ! File version
integer:: species(1:num_popn_orb)! Atomic species associated with each projector
integer:: ion(1:num_popn_orb)  ! Ion associated with each projector
integer:: am_channel(1:num_popn_orb) ! Angular momentum channel
integer:: num_eigenvalues(1:num_spins) ! Number of eigenvalues per spin channel
real(dp):: kpoint_positions(1:num_kpoints,1:3) ! k_x, k_y, k_z in fractions of BZ
real(dp):: pdos_weights(1:num_popn_orb,max_eigenv,num_kpoints,num_spins)!Matrix elements
character(len=80):: file_header ! File header comment

write(pdos_file) file_header
write(pdos_file) file_version
write(pdos_file) num_kpoints
write(pdos_file) num_spins
write(pdos_file) num_popn_orb
write(pdos_file) max_eigenv
write(pdos_file) species(1:num_popn_orb)
write(pdos_file) ion(1:num_popn_orb)
write(pdos_file) am_channel(1:num_popn_orb)
```

```
do nk=1,num_kpoints
  write(pdos_file) nk, kpoint_positions(nk,:)
  do ns = 1,num_spins
    write(pdos_file) ns
    write(pdos_file) num_eigenvalues(ns)
    do nb = 1,num_eigenvalues(ns)
      write(pdos_file) real(pdos_weights(num_popn_orb,nb,nk,ns))
    end do
  end do
end do
```

A.4 .elnes_bin file

The .elnes_bin file is required for ELNES calculations. The .elnes_bin file is an unformatted file.

```
integer,parameter:: dp=selected_real_kind(15,300) ! Define double precision
integer:: tot_core_projectors ! Total number of core states included in matrix elements
integer:: max_eigenvalues ! Number of bands included in matrix elements
integer:: num_kpoints ! Number of k-points
integer:: num_spins ! Number of spins
integer:: num_eigenvalues(1:num_spins) ! Number of eigenvalues
integer:: species(1:tot_core_projectors) ! Atomic species associated with each projector
integer:: ion(1:tot_core_projectors) ! Ion associated with each projector
integer:: n(1:tot_core_projectors) ! Principal quantum number associated with each projector
integer:: lm(1:tot_core_projectors) ! Angular momentum quantum number associated with each projector
complex(dp):: elnes_mat(tot_core_projectors,max_eigenvalues,3,num_kpoints,num_spins) ! matrix elements

write(elnes_file) tot_core_projectors
write(elnes_file) max_eigenvalues
write(elnes_file) num_kpoints
write(elnes_file) num_spins
write(elnes_file) species(1:tot_core_projectors)
write(elnes_file) ion(1:tot_core_projectors)
write(elnes_file) n(1:tot_core_projectors)
write(elnes_file) lm(1:tot_core_projectors)

do nk = 1,num_kpoints
do ns = 1, num_spins
write(elnes_file) (((elnes_mat(orb,nb,indx,nk,ns),orb=1,&
&tot_core_projectors),nb=1,num_eigenvalues(ns)),indx=1,3)
end do
end do
```

A.5 .dome_bin file

The .dome_bin file contains the first derivative of a band with respect to k . Also known as electron velocity.

The .dome_bin file is an unformatted file.

```
integer,parameter:: dp=selected_real_kind(15,300) ! Define double precision
real(dp):: file_version=1.0_dp ! File version
character(len=80):: file_header ! File header comment
integer:: num_kpoints ! Number of k-points
integer:: num_spins ! Number of spins
integer:: max_eigenv ! Number of bands included in matrix elements
integer:: num_eigenvalues(1:num_spins) ! Number of eigenvalues per spin channel
real(dp):: band_gradient(max_eigenv,1:3,1:num_kpoints,num_spins)!

write(gradient_unit) file_version
write(gradient_unit) file_headerphoto_gkgrid

do ik=1,num_kpoints
do is=1,num_spins
write(gradient_unit) ((band_gradient(ib,i,ik,is),ib=1,num_eigenvalues(is)),i=1,3)
end do
end do
```

A.6 .fem_bin file

The .fem_bin file is required for the one step model. The .fem_bin file is an unformatted file.

```
integer,parameter:: dp=selected_real_kind(15,300) ! Define double precision
real(dp):: file_version=1.0_dp ! File version
character(len=80):: file_header ! File header comment
integer:: num_kpoints ! Number of k-points
integer:: num_spins ! Number of spins
integer:: max_eigenv ! Number of bands included in matrix elements
integer:: num_eigenvalues(1:num_spins) ! Number of eigenvalues per spin channel
! Information on: (num_energy_step, min_e_photon, size_energy_step,
! assumed E_Fermi, assumed E_workfct)
! ATTENTION: E_Fermi must either match the OptaDOS computed E_Fermi or be set to the same value
! ATTENTION: E_workfct must be set to the same value during creation of fem_bin file and photoemission run
real(dp):: fem_energy_info(5)
! FEMs
complex(dp):: foptical_mat(max_eigenv,1:3,1:num_energy_step,1:num_kpoints,num_spins)

write(fem_unit) file_version
write(fem_unit) file_header
write(fem_unit) fem_energy_info(1:5)

do ik=1,num_kpoints
do is=1,num_spins
write(fem_unit) (((foptical_mat(ib,i,ne,ik,is),ib=1,num_eigenvalues(is)),&
i=1,3),ne=1,num_energy_step)
end do
end do
```

A.7 .tmprob_bin file

The .tmprob_bin file is required for the three step model if photo_use_tmprob is True. The .tmprob_bin file is an unformatted file.

```
integer,parameter:: dp=selected_real_kind(15,300) ! Define double precision
real(dp):: file_version=1.0_dp ! File version
character(len=80):: file_header ! File header comment
integer:: num_kpoints ! Number of k-points
integer:: num_spins ! Number of spins
integer:: max_eigenv ! Number of bands included in tmprobs
integer:: num_eigenvalues(1:num_spins) ! Number of eigenvalues per spin channel
complex(dp):: transmit_prob(max_eigenv, num_spins,1:num_kpoints)! OMEs

write(tmprob_unit) file_version
write(tmprob_unit) file_header

do ik=1,num_kpoints
  do is=1,num_spins
    write(tmprob_unit) (transmit_prob(ib,is,ik),ib=1,num_eigenvalues(is))
  end do
end do
```

A.8 .gkgrid_bin file

The .gkgrid_bin file is required if photo_momentum is set to gkgrid. The .gkgrid_bin file is an unformatted file.

```
integer,parameter:: dp=selected_real_kind(15,300) ! Define double precision
real(dp):: file_version=1.0_dp ! File version
character(len=80):: file_header ! File header comment
integer:: num_kpoints ! Number of k-points
integer:: num_spins ! Number of spins
integer:: max_eigenv ! Number of bands included in tmprobs
integer:: max_gkgrid ! Number of grid points
integer:: num_eigenvalues(1:num_spins) ! Number of eigenvalues per spin channel
complex(dp):: photo_gkgrid(3,max_gkgrid,max_eigenv,num_spins,num_kpoints)! OMEs

write(gkgrid_unit) file_version
write(gkgrid_unit) file_header

do ik=1,num_kpoints
do is=1,num_spins
write(gkgrid_unit) ((photo_gkgrid(i,gdx,ib,is,ik),i=1,3),gdx=1,max_gkgrid),&
ib=1,num_eigenvalues(is))
end do
end do
```

Bibliography

- [1] C. Ambrosch-Draxl and J.O. Sofo. Linear optical properties of solids within the full-potential linearized augmented planewave method. *Comput. Phys. Commun.*, 175:1, 2006.
- [2] P. E. Blöchl, O. Jepsen, and O. K. Andersen. Improved tetrahedron method for brillouin-zone integrations. *Phys. Rev. B*, 49(23):16223, 1994.
- [3] M. Boon, M. Methfessel, and F. Mueller. Singular integrals over the brillouin zone: the analytic-quadratic method for the density of states. *J. Phys. C*, 19:5337, 1986.
- [4] M. Dressel and G. Grüner. *Electrodynamics of Solids*. C.U.P., 2002.
- [5] R.F. Egerton. *Electron Energy-Loss Spectroscopy in the Electron Microscope*. Plenum Press, 1996.
- [6] Richard G. Forbes and Jonathan H. B. Deane. Reformulation of the standard theory of fowler–nordheim tunnelling and cold field electron emission. *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 463(2087):2907–2927, 2007.
- [7] G. Lehmann and M. Taut. On the numerical calculation of the density of states and related properties. *Phys. Stat. Sol. (b)*, 54:469, 1972.
- [8] M. Methfessel, M. Boon, and F. Mueller. Analytic-quadratic method of calculating the density of states. *J. Phys. C*, 16:1949, 1983.
- [9] M. Methfessel, M. Boon, and F. Mueller. Singular integrals over the brillouin zone: inclusion of k-dependent matrix elements. *J. Phys. C*, 20:1069, 1987.
- [10] Teruyasu Mizoguchi, Isao Tanaka, Shang-Peng Gao, and Chris J Pickard. First-principles calculation of spectral features, chemical shift and absolute threshold of ELNES and XANES using a plane wave pseudopotential method. *Journal of Physics: Condensed Matter*, 21(10):104204, feb 2009.
- [11] H.J. Monkhorst and J.D. Pack. Special points for brillouin-zone integrations. *Phys. Rev. B*, 13:5188, 1976.
- [12] A. J. Morris, C. J. Pickard, and R. J. Needs. Hydrogen/silicon complexes in silicon from computational searches. *Phys. Rev. B*, 78:184102, 2008.
- [13] J. E. Müller and J. W. Wilkins. Band-structure approach to the x-ray spectra of metals. *Phys. Rev. B*, 8:4331, 1984.

-
- [14] I. Nagy and P. M. Echenique. Mean free path of a suddenly created fast electron moving in a degenerate electron gas. *Physical Review B*, 85(11):115131, 2012.
 - [15] L. W. Nordheim. The effect of the image force on the emission and reflexion of electrons by metals. *Proceedings of the Royal Society of London. Series A, Containing Papers of a Mathematical and Physical Character*, 121(788):626–639, 1928.
 - [16] C. J. Pickard and M. C. Payne. Extrapolative approaches to brillouin-zone integration. *Phys. Rev. B*, 59(7):4685, 1999.
 - [17] C. J. Pickard and M. C. Payne. Second-order k -perturbation theory with vanderbilt pseudopotentials and plane waves. *Phys. Rev. B*, 62(7):4383, 2000.
 - [18] G. Rajagopal, R. J. Needs, S. Kenny, W. M. C. Foulkes, and A. James. Quantum monte carlo calculations for solids using special k points methods. *Phys. Rev. Lett.*, 73:1959, 1994.
 - [19] K. Refson. `orbitals2bands` program. see www.castep.org.
 - [20] Pallavi Saha, Oksana Chubenko, J. Kevin Nangoi, Tomas Arias, Eric Montgomery, Shashi Poddar, Howard A. Padmore, and Siddharth Karkare. Theory of photoemission from cathodes with disordered surfaces. *Journal of Applied Physics*, 133(5), 2023.
 - [21] M. D. Segall, C. J. Pickard, R. Shah, and M. C. Payne. Population analysis in plane wave electronic structure calculations. *Mol. Phys.*, 89(2):571, 1996.
 - [22] E. W. Tait, L. E. Ratcliff, M. C. Payne, P. D. Haynes, and N. D. M. Hine. Simulation of electron energy loss spectra of nanomaterials with linear-scaling density functional theory. *J. Phys: Condens Matter*, 2016.
 - [23] J. Van Hove. The occurrence of singularities in the elastic frequency distribution of a crystal. *Phys. Rev.*, 89(6):1189, 1953.
 - [24] J. R. Yates, X. Wang, D. Vanderbilt, and Souza I. Spectral and fermi surface properties from wannier interpolation. *Phys. Rev. B*, 75:195121, 2007.
 - [25] Oleg V. Yazyev, Konstantin N. Kudin, and Gustavo E. Scuseria. Efficient algorithm for band connectivity resolution. *Phys. Rev. B*, 65:205117, 2002.